Mohsin Iban Hossain

AIUB, Operating System Notes

Table of Contents

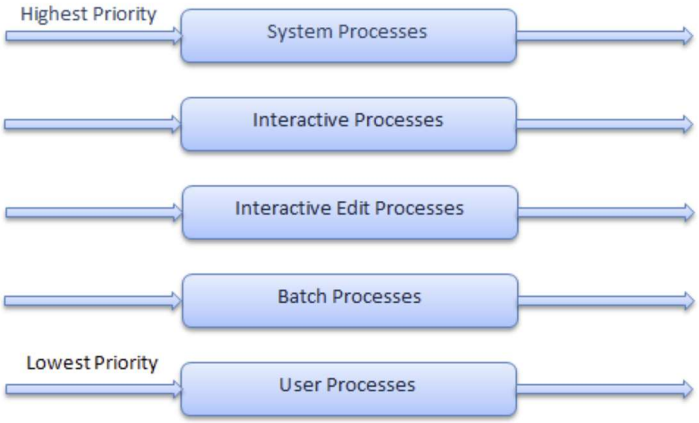
CPU SCHEDULING

Multilevel Queue Scheduling

- Ready queue is divided into **multiple queues**.
- Each queue has a **different priority**.
- CPU always selects the **highest-priority non-empty queue**.
- Each queue uses its **own scheduling algorithm**.

Note: Processes **cannot move** between queues.

Prioritization based upon process type



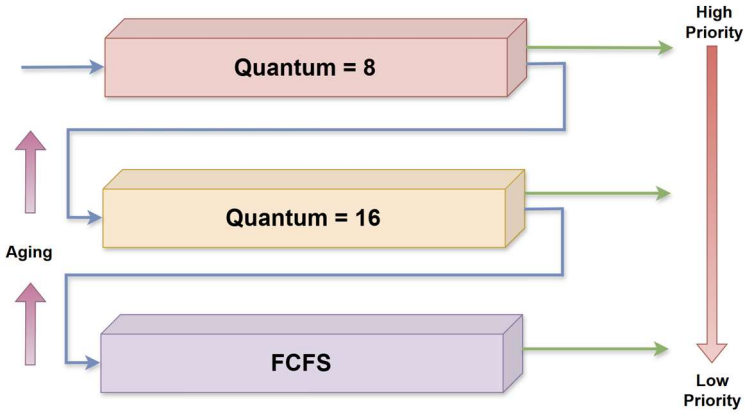
Multilevel Feedback Queue

- The multilevel feedback-queue scheduling algorithm allows a **process to move** between queues
- Extension of multilevel queue.
- The idea is If a process **uses too much CPU time**, it will be **moved to a lower-priority queue**.
- If, **a process that waits too long in a lower-priority queue** may be **moved to a higher-priority queue**. This form of aging prevents starvation.

Defined by the following parameters:

1. number of queues
2. scheduling algorithms for each queue
3. method used to determine when to upgrade a process
4. method used to determine when to demote a process
5. method used to determine which queue a process will enter when it needs service

➢ **Example:** there are three queues



- **Q0:** Round Robin (8 ms)
- **Q1:** Round Robin (16 ms)
- **Q2:** FCFS

❖ **Scheduling** A new job starts in **Q0 (RR)** and gets **8 ms** of CPU time. If unfinished, it moves to **Q1 (RR)** and gets **16 ms** more. If still not finished, it is **preempted and moved to Q2**.

Thread Scheduling

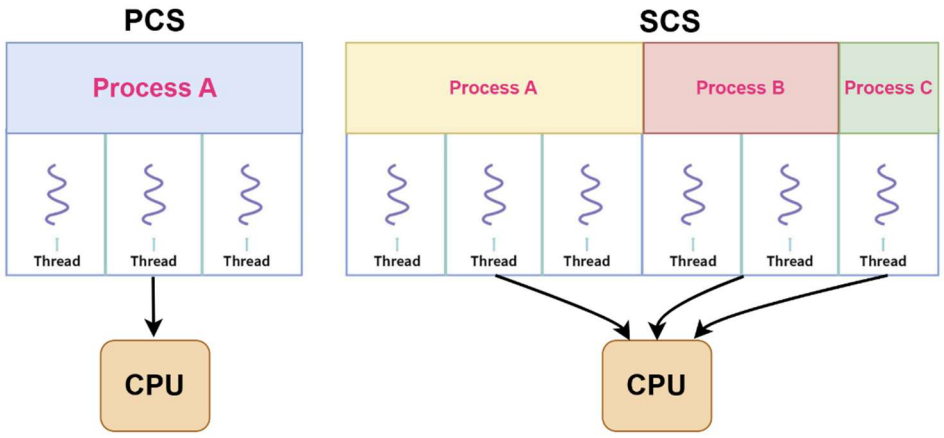
⇨ When threads exist, **threads are scheduled, not processes**.

❖ **Thread Types:**

1. User-level threads
2. Kernel-level threads

➢ **Scheduling Scopes:**

1. **PCS (Process Contention Scope)**
   - Competition among threads of same process
2. **SCS (System Contention Scope)**
   - Competition among all system threads



Pthread Scheduling

⇨ **Pthread (POSIX thread)** is a **standard API** used to create and manage **threads** in a program on **Unix-like operating systems**.

❖ Thread Types:

1. **PTHREAD_SCOPE_PROCESS** → use **PCS**
    - Threads compete **only within the same process** for the CPU.
2. **PTHREAD_SCOPE_SYSTEM** → use **SCS**
    - Threads compete **with all threads in the system**, across all processes.

➢ **Note: Linux and macOS support only SCS**, meaning all pthreads are scheduled by the **kernel at the system level**.

Multiple-Processor Scheduling

⇨ **Multiple-processor scheduling** is more complex because the operating system must decide **which CPU** a process or thread should run on.

❖ Architectures:

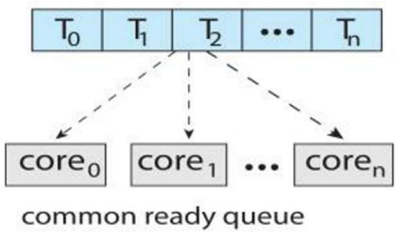
- Multicore CPUs
- Multithreaded cores
- NUMA systems
- Heterogeneous multiprocessing

Symmetric Multiprocessing (SMP)

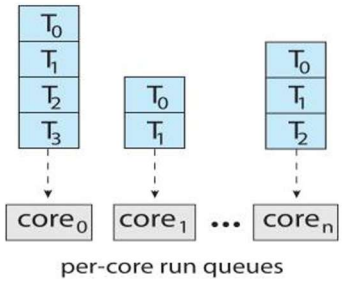
⇨ A system with multiple identical processors sharing the same memory and I/O. All processors are equal and can run tasks independently.

➢ Two Models of Ready Queues:

1. **Common Ready Queue**
    - All CPUs share one queue of ready processes.
    - **Pros:** Easy load balancing; keeps all CPUs busy.
    - **Cons:** Needs synchronization (locks) for shared access.



2. **Separate Ready Queue per CPU**
    - Each CPU has its own queue.
    - **Pros:** Less contention; simpler.
    - **Cons:** Load may be uneven; some CPUs idle while others are busy.

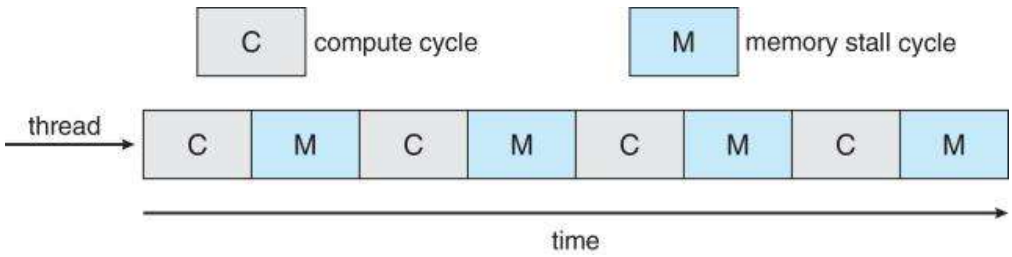


Multicore Processors

⇨ A **multicore processor** has **two or more processing cores** on a single chip. Each core can execute instructions independently, allowing **parallel processing**.

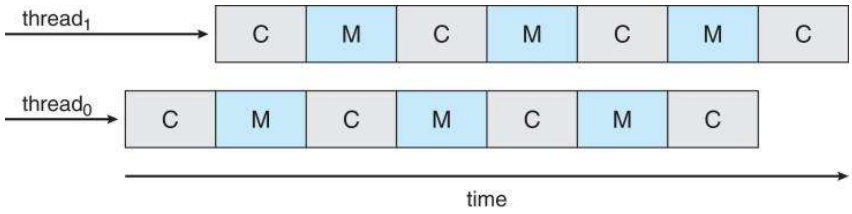
❖ Recent Trends in Multicore Processors

- **Multiple cores on a single chip:** Modern processors place several cores on the **same physical chip**.
- **Faster and energy-efficient:** Multicore chips **run faster** and **consume less power** than multiple separate CPUs.
- **Multiple threads per core:** Each core can run **multiple threads** simultaneously.
- **Memory stall optimization:** When one thread waits for memory, the processor can **switch to another thread** to keep the core busy.
- **Goal:** Improve **performance** and **resource utilization** while reducing **power consumption**.



Multithreaded Multicore System

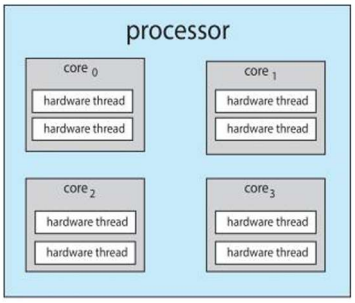
- ⇨ Each **core** can run **more than one hardware thread**.
- ⇨ If **one thread is waiting** for memory (memory stall), the core **switches to another thread**.
- ⇨ This keeps the core **busy all the time**, improving **performance** and **efficiency**.



Multithreaded Multicore System

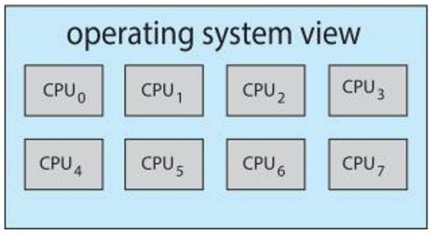
❖ Chip-Multithreading (CMT)

- CMT allows a single CPU core to handle **multiple hardware threads** simultaneously.
- **Intel Terminology:** Intel calls this **Hyper-Threading (HT)**.
- **Purpose:** Improves CPU utilization by allowing the core to work on another thread when one thread is stalled (e.g., waiting for memory).



❖ Logical vs Physical Processors

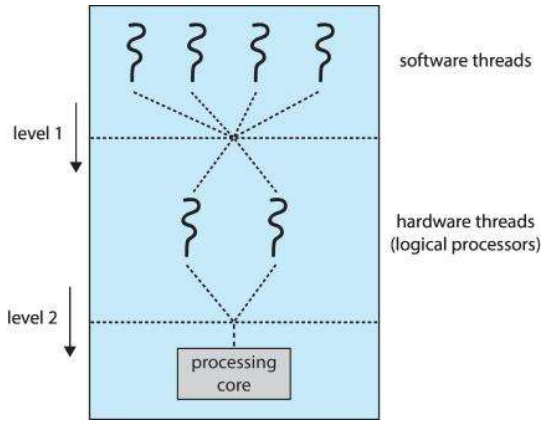
- **Example:** A **quad-core CPU** with **2 hardware threads per core**.
- **Observation by OS:** The OS sees **8 logical processors** (4 cores × 2 threads).
- **Implication:** The OS schedules software threads across **logical processors**, not physical cores directly.



❖ Two Levels of Scheduling

1. **Operating System Level:**
   - The OS decides **which software thread** to run on **which logical processor**.
   - Treats each logical processor independently.

2. **Core Level (Hardware Scheduling):**
   - Each core decides **which hardware thread** (among the threads mapped to it) gets the execution resources.
   - Ensures the core stays busy even if one thread is stalled.



Load Balancing Techniques

⇨ In **Symmetric Multiprocessing (SMP)**, all CPUs are equal and share the same memory.

⇨ **Efficiency requires keeping all CPUs busy**.

⇨ **Load balancing** ensures the workload is **evenly distributed** across CPUs.

❖ Two main approaches for moving tasks between CPUs:

a) Push Migration

- **Mechanism:** Each CPU periodically checks its own load.
- If a CPU is **overloaded**, it **pushes some tasks** to less busy CPUs.
- **Analogy:** "I'm too busy, I'm offloading work to others."

b) Pull Migration

- **Mechanism:** Idle CPUs look for work.
- An idle CPU **pulls tasks** from a busy CPU's queue.
- **Analogy:** "I'm free, let me grab some work from someone busy."

Processor Affinity

⇨ When a thread repeatedly runs on the **same processor**, the processor's cache hold its recently used data and instructions.

❑ **Affinity:** The thread has a **"preference"** for that processor because keeping it there improves cache performance.

➢ Interaction with Load Balancing

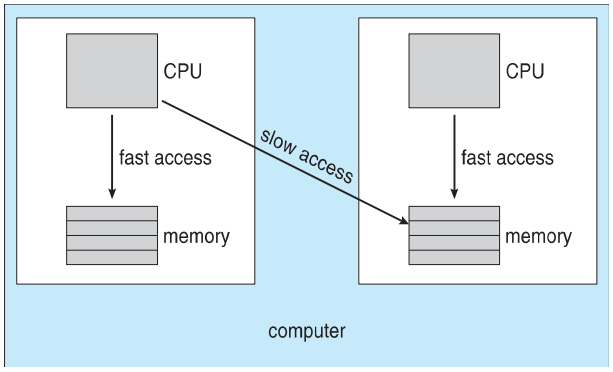
- **Challenge:** Load balancing may move threads to other processors to even out CPU usage.
- **Problem:** When a thread moves:
  - It **loses the cache contents** stored on the original processor.
  - This can **increase memory access time** until the cache warms up again.

❖ Types of Processor Affinity:

Type	Description
Soft Affinity	OS **tries** to keep a thread on the same processor but can move it if needed.
Hard Affinity	A process specifies the exact set of processors it can run on; OS **cannot move it** outside that set.

NUMA and CPU Scheduling

⇨ **NUMA (Non-Uniform Memory Access)** is a memory architecture where **memory access time depends on the memory's location relative to the CPU**.
⇨ **Local memory:** Memory physically close to a CPU is **faster to access**.
⇨ **Remote memory:** Memory on another CPU's node is **slower to access**.



➢ NUMA-Aware Scheduling

⇨ A **NUMA-aware OS system** assigns memory **close to CPU**
⇨ Reduces memory access delay.

Real-Time CPU Scheduling

⇨ **Real-time scheduling** is used when tasks have **time constraints**.
⇨ Challenges: The CPU must respond **quickly and predictably** to events.

❖ Types of Real-Time Systems:

Type	Description
Soft Real-Time	Critical tasks have **highest priority**, but **no absolute guarantee** of meeting deadlines. Missing a deadline **degrades performance**, but system still works.
Hard Real-Time	Tasks **must be completed by their deadline**; missing a deadline can cause system failure. Scheduler must guarantee **deadline satisfaction**

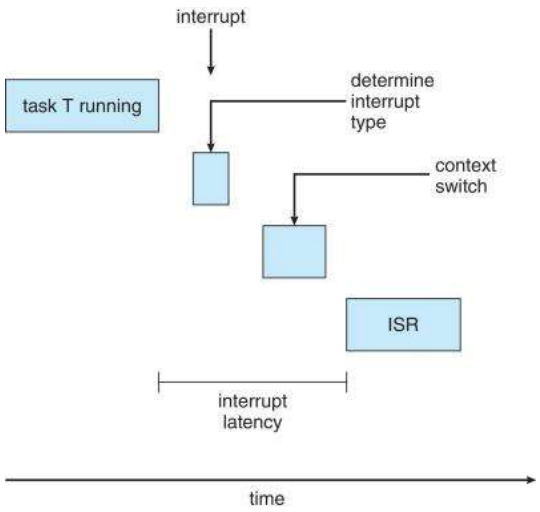
Event Latency

⇨ Time from when an **event occurs** to when it is **actually served**.

❖ Two main types of event latency affect real-time performance:

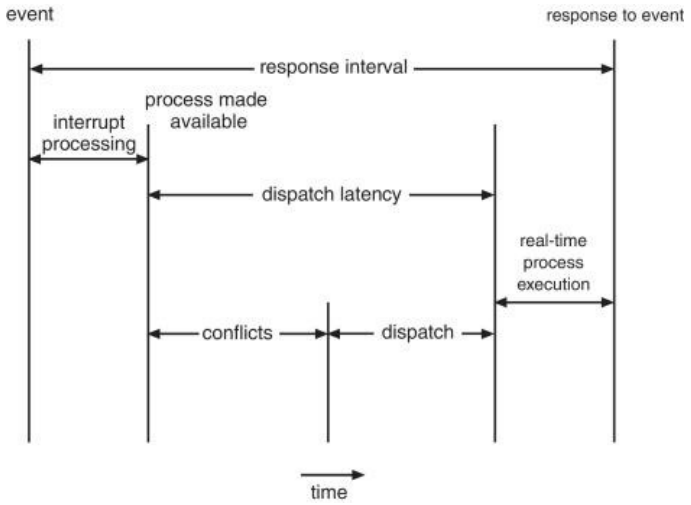
➢ Interrupt Latency:

⇨ Time from **arrival of an interrupt** to the **start of the interrupt service routine (ISR)**. Affected by CPU load, masking, and OS interrupt handling.



➢ Dispatch Latency:

⇨ Time taken for the **scheduler to preempt the current process** and switch the CPU to a **ready process**. Includes context-switch overhead.



❖ Conflict Phase of Dispatch Latency:

- Delay occurs when a **high-priority process cannot run immediately**.
- Happens if:
  1. A **low-priority process is executing in kernel mode** and cannot be preempted.
  2. A **low-priority process holds resources** needed by the high-priority process.
- This increases dispatch latency and can cause **priority inversion**.

Priority-Based Scheduling

⇨ Real-time schedulers must use **preemptive, priority-based scheduling**. This **guarantees only soft real-time** behavior.

⇨ **Hard real-time systems** additionally require **deadline guarantees**.

➢ Characteristics of Real-Time Processes:

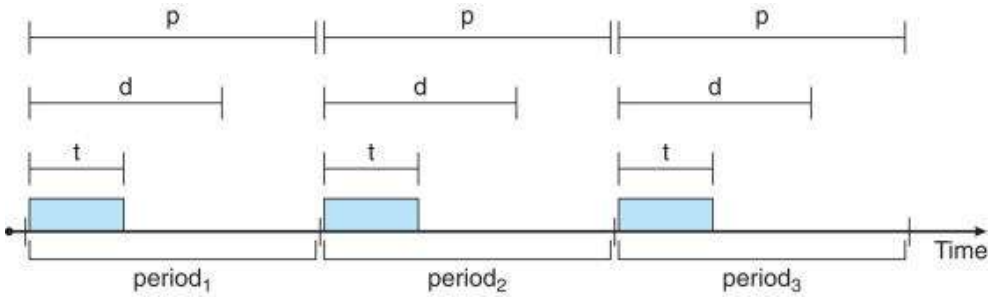
⇨ **Real-time processes have** time constraints**, not just execution order.**

➢ Periodic Processes:

- Tasks that repeat at **regular intervals**.
- Require CPU at constant time periods.
- Each periodic task has:
  - **Processing Time (t)**: Time required to complete the task.
  - **Deadline (d)**: Time by which the task must finish.
  - **Period (p)**: Time interval between two consecutive executions.
  - **Constraint:** $0 \leq t \leq d \leq p$

- Rate of a Periodic Task
  - Rate indicates how often a task executes.
  - **Rate = 1 / p**
  - Smaller period ⇒ **higher rate ⇒ higher priority** (in rate-based scheduling).

- **Periodic Processes Diagram:**



Algorithm Evaluation

➢ Purpose of Algorithm Evaluation:

- To **select the best CPU-scheduling algorithm** for an operating system.
- First, **define evaluation criteria**, then **compare different algorithms**.
- goal: improve system performance.

➢ Evaluation Criteria:

⇨ A CPU-scheduling algorithm is evaluated based on:

- **CPU utilization**
- **Throughput**
- **Turnaround time**
- **Waiting time**
- **Response time**

➢ Analytic Evaluation:

- Uses **mathematical or logical analysis**.
- Does **not require implementation**.
- One important analytic method is **Deterministic Modeling**.

➢ Deterministic Modeling:

- A type of **analytic evaluation**.
- Uses a **specific, predetermined workload**.
- Calculates the **performance of each scheduling algorithm** for that workload.
- Performance is usually measured by **average waiting time**.

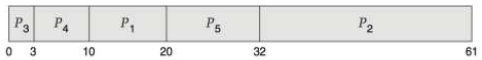
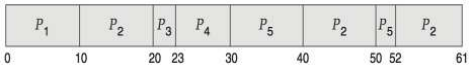
➢ Deterministic Evaluation (Example):

- Consider 5 processes arriving at time 0:

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12

1. Apply each scheduling algorithm.
2. Calculate **average waiting time**.
3. Choose the algorithm with the **minimum average waiting time**.

Results of Deterministic Evaluation

Scheduling Algorithm	Average Waiting Time	Gantt chart
FCFS	**28 ms**	
Non-preemptive SJF	**13 ms**	
Round Robin (RR)	**23 ms**	

➢ **So:**

- **Non-preemptive SJF** gives the **minimum average waiting time**.
- Hence, SJF performs best **for this specific workload**.

Advantages of Deterministic Modeling

- **Simple and fast**
- Easy to compare algorithms
- Useful for **theoretical understanding**

Limitations of Deterministic Modeling

- Requires **exact input values**
- Results apply **only to that workload**
- Not suitable for real-world dynamic systems

Queueing Models

➢ Purpose of Queueing Models:

- Queueing models describe:
  - **Arrival of processes**
  - **CPU and I/O burst times**
- These are described **probabilistically**, not exactly.
- Most commonly use **exponential distributions**.
- Characterized using **mean (average) values**.

➢ What Queueing Models Compute:

⇨ Queueing models help compute:

- **Average throughput**
- **CPU utilization**
- **Average waiting time**
- **Average queue length**

➢ System Representation:

- A computer system is modeled as a **network of servers**.
- Each server has:
  - A **queue of waiting processes**
- Examples of servers:
  - CPU
  - Disk
  - I/O devices

➢ Arrival and Service Rates:

- **Arrival rate (λ):** average number of processes arriving per unit time.
- **Service rate (μ):** average number of processes served per unit time.
- Knowing arrival and service rates allows calculation of:
  - **Utilization**
  - **Average queue length**
  - **Average waiting time**

Little's Formula

$$n = \lambda \times W$$

- **Where,**
  - **n** = average number of processes in the queue
  - **W** = average waiting time in the queue
  - **λ** = average arrival rate into the queue

- **Example:**
  - If, Average arrival rate, λ = **7 processes/second &**
  - Average queue length, n = **14 processes**

$$W = \frac{n}{\lambda} = \frac{14}{7} = 2sec$$

∴ Average waiting time per process = 2 seconds

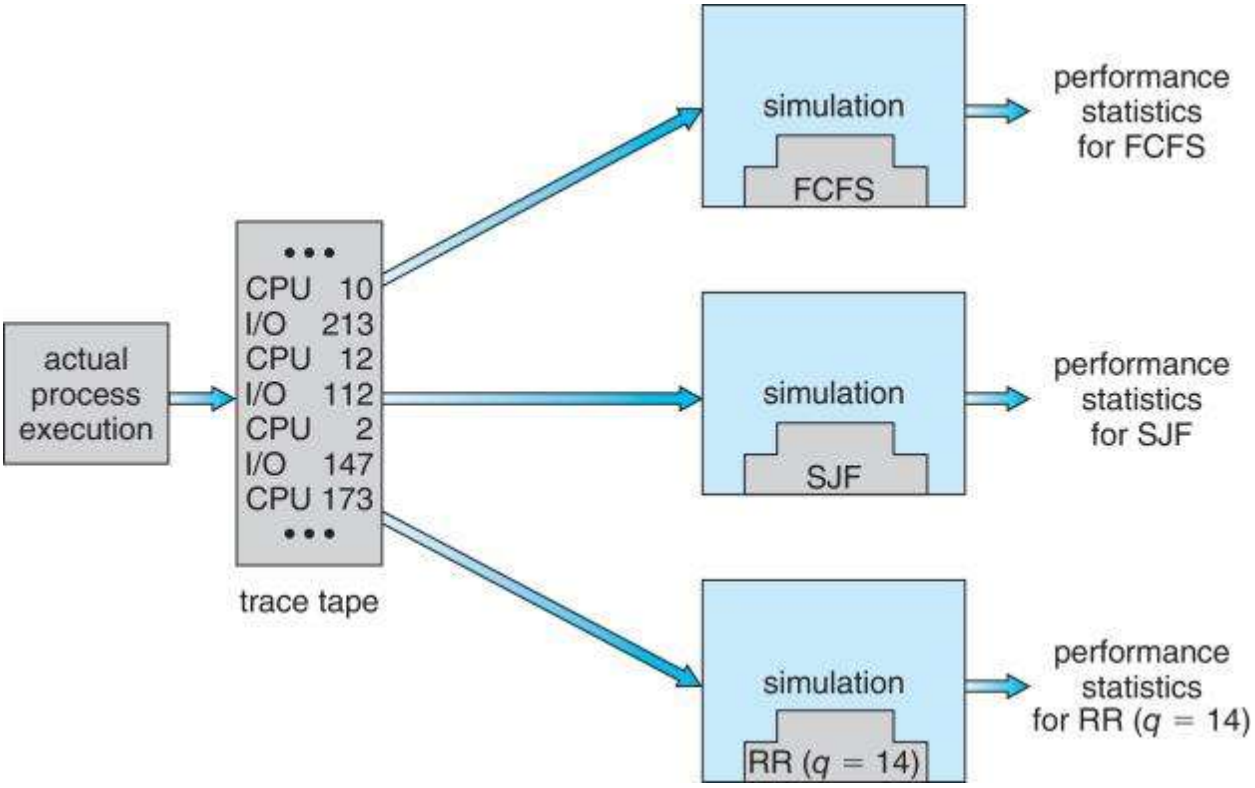
- **Key Properties:**
  - Valid in **steady state** (system is stable).
  - Processes leaving the queue = processes arriving.
  - Applicable to:
    - **Any scheduling algorithm**
    - **Any arrival distribution**

Simulations (CPU Scheduling Evaluation)

- **Queueing models** are limited in accuracy.
- **Simulations** provide more accurate results than analytic models.
- A **programmed model** of the computer system is used.
- **Clock** acts as a variable to simulate time progression.
- The simulator **gathers statistics** to measure algorithm performance.

- **Input Data for Simulation:**
  - Generated using **random number generators** based on probability distributions.
  - Distributions may be:
    - **Mathematically defined**
    - **Empirically measured**
  - **Trace tapes**: records of real events from actual systems, used to drive realistic simulations.

Evaluation of CPU Schedulers by Simulation



Implementation (CPU Scheduling Evaluation)

- Even **simulations** have limited accuracy.
- The most direct method is to **implement the scheduler and test it on real systems**.
- **Disadvantages:**
  - High **cost** and **risk**
  - System **environments vary**, so results may differ
- Many modern schedulers are **flexible**:
  - Can be **tuned per system or per site**
  - Provide **APIs to adjust priorities**
- Still, **environmental differences** make universal evaluation difficult.

SYNCHRONIZATION TOOLS

PART:1

Overview of Process Synchronization

➔ **Process Synchronization** is a mechanism in an operating system that controls the execution of multiple processes that share common resources (such as memory, files, or variables). Its main goal is to **maintain data consistency** by ensuring that **only one process accesses or modifies shared data at a time**.

To achieve synchronization, the OS uses techniques like **semaphores, mutex locks, monitors, and hardware-based solutions**.

Background of Synchronization

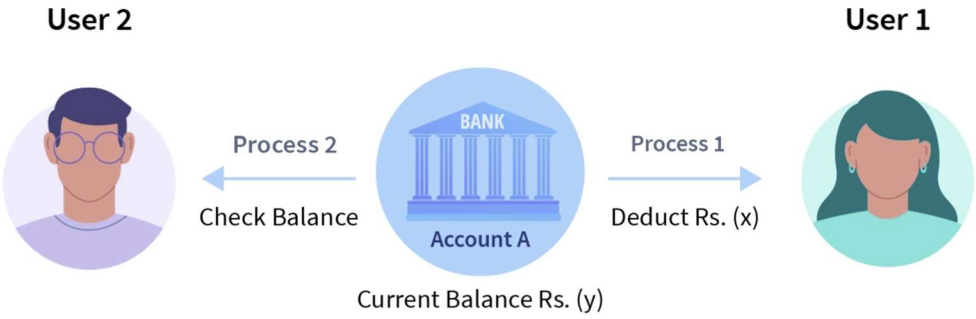
- ⇨ In an **Operating System**, multiple **processes** can run **at the same time**.
- ⇨ These processes may:
  - Share **common data**
  - Be **interrupted** at any moment
- ⇨ If two processes change shared data **simultaneously**, **wrong results** can occur. Means it can lead to **data inconsistency**.

❖ **Note:** To avoid this, we need **synchronization mechanisms**.

❖ **Example: Consider a** bank database**:**

- Initial balance = **X**
- Process 1: Withdraws money
- Process 2: Reads account balance at the same time

Since the withdrawal process takes time, Process 2 may read the old balance **X** instead of the updated balance **Y**, leading to **incorrect information**. This problem occurs because both processes access shared data **simultaneously**.



Producer–Consumer Problem Example

❖ Problem Idea

- A **Producer** produces items and puts them into a buffer.
- A **Consumer** consume item & removes items from the buffer.
- A **shared variable** `counter` tracks how many items are in the buffer.

Shared Variable

```c
#define BUFFER_SIZE 5
int buffer[BUFFER_SIZE];
int counter = 0; // Shared Variable
int in = 0;
int out = 0;
```

Producer Code

```c
void producer() {
    int item;

    while (true) {
        item = produce_item();
        // produce an item

        while (counter == BUFFER_SIZE);
        // wait if buffer is full

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
    }
}
```

Consumer Code

```c
void consumer() {
    int item;

    while (true) {
        while (counter == 0);
        // wait if buffer is empty

        item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        counter--;

        consume_item(item);
        // consume the item
    }
}
```

➢ **Problem Arise:** when both access `counter` at the same time → **Race Condition**

Race Condition

⇨ A **race condition** occurs when:

- Two or more processes access **shared data**
- They **modify it at the same time**
- Final result depends on **execution order**

`counter++` could be implemented in cpu as

```
Step 1: register1 = counter
Step 2: register1 = register1 + 1
Step 3: counter = register1
```

`Counter--` could be implemented in cpu as

```
Step 1: register2 = counter
Step 2: register2 = register2 - 1
Step 3: counter = register2
```

❖ Example Execution (Wrong Result):

Initial value

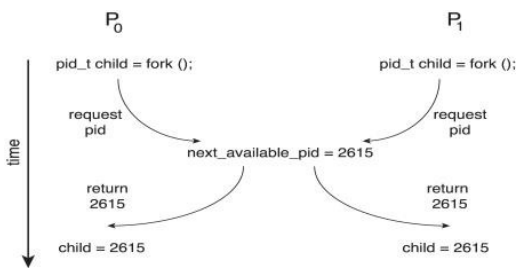
```
counter = 5
```

Step	Process	Action	Result
S0	Producer	register1 = counter	5
S1	Producer	register1++	6
S2	Consumer	register2 = counter	5
S3	Consumer	register2--	4
S4	Producer	counter = register1	counter = 6
S5	Consumer	counter = register2	counter = 4
This is wrong because Final value should be **5**, but becomes **4**			

❖ Another Race Condition Example:

- OS assigns **process IDs (PID)** using `next_available_pid`
- Two processes call `fork()` at the same time
- Both may receive the **same PID**

➢ Note: This causes **serious OS errors**



Critical Section Problem

⇨ The **Critical Section Problem** deals with controlling access to shared resources so that **only one process can access shared data at a time**.

⇨ A **critical section** is a part of a program where shared data is accessed or modified.

❖ Why is it needed?

- To prevent **race conditions**
- To maintain **data consistency**
- To ensure **correct program execution**

❖ Entry & Exit

- `wait()` → controls entry to the critical section
- `signal()` → controls exit from the critical section

Solution to Critical-Section Problem

⇨ Any solution to the critical section problem **must satisfy all three conditions**:

1. Mutual Exclusion

- Only **one process** can be in the critical section at a time.

2. Progress

- If no process is in the critical section, one of the waiting processes must be allowed to enter **without unnecessary delay**.

3. Bounded Waiting

- A process should not wait **forever** to enter the critical section.
- There must be a limit on the number of times other processes can enter before it.

Sections of a Program in Process Synchronization

1. Entry Section

- Determines whether a process can enter the **critical section**
- Includes synchronization code (e.g., lock checking)

2. Critical Section

- The part of the program where **shared data is accessed or modified**
- **Only one process** is allowed here at a time

3. Exit Section

- Executes after the critical section
- Releases the lock and allows other processes to enter

4. Remainder Section

- The rest of the program that **does not access shared resources**
- Can be executed without synchronization

Critical Section Handling in OS

⇨ Two approaches depending on if kernel is preemptive or non-preemptive

❖ **Preemptive Kernel**

- Process can be interrupted in kernel mode
- More **chances of race conditions**

❖ **Non-Preemptive Kernel**

- Process runs until:
  - It finishes
  - It blocks
  - It yields CPU
- Almost **race-condition free**

Peterson's Solution

⇨ **Peterson's Solution** is a **classical software-based solution** to the **critical section problem**. It ensures that **only one of two processes** can enter the critical section at a time, thus preventing race conditions.

➢ **Peterson's solution** is used to control **two processes only**.

➢ The processes are named P_i and P_j.

➢ **Shared Variables:**

⇨ To work properly, Peterson's solution uses **two shared variables**:

- `boolean Flag[2]:` use to indicate if a process is ready to enter its critical section.
- `int Turn:` Indicates whose turn it is to enter the critical section.

Structure of process Pi in Peterson's solution	Structure of process Pj in Peterson's solution

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);

    critical section

    flag[i] = false;

    remainder section
} while (true);
```

```
do {
    flag[j] = true;
    turn = i;
    while (flag[i] && turn == i);

    critical section

    flag[j] = false;

    remainder section
} while (true);
```

Why Peterson's Solution Works

✓ **Mutual Exclusion**
✓ **Progress**
✓ **Bounded Waiting**

➢ But **Not safe on modern systems**

Why Peterson's Solution Fails

⇨ Although **Peterson's Solution** is useful for understanding synchronization, it is **not guaranteed to work on modern processors** due to **instruction reordering** by compilers and CPUs.

❖ Instruction Reordering:

⇨ To improve performance, **compilers and processors may reorder instructions** that appear independent.

- This is **safe for single-threaded programs**
- But **dangerous for multithreaded programs**, as it may cause **unexpected or inconsistent results**

Example Scenario

Shared Variable

```
boolean flag = false;
int x = 0;
```

Thread 1	Thread 2
`while (!flag);` `print(x);`	`x = 100;` `flag = true;`

➢ **Expected Output:** `100` (since `x` is set before `flag` becomes true)

❖ What Can Go Wrong?

⇨ Due to instruction reordering, Thread 2 may execute as:

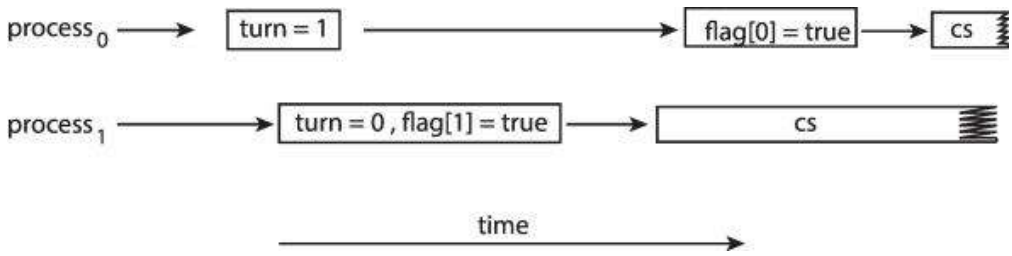
Thread 2

```
flag = true;
x = 100;
```

➢ **Now:** ○ Thread 1 sees `flag == true` ○ Immediately prints `x` ○ But `x` is still `0`	**Output** becomes `0` instead of `100`

Impact on Peterson's Solution

- Memory reordering breaks the assumptions of Peterson's algorithm
- Mutual exclusion may fail
- **Both processes may enter the critical section simultaneously**
- Leads to **race conditions**



➢ This allows both processes to be in their critical section at the same time!

SYNCHRONIZATION TOOLS

PART:2

Synchronization Hardware Support

⇨ Many computer systems provide **hardware-level support** to help implement **critical sections**, so that only one process can access shared data at a time.

❖ Uniprocessor System:

- The system can **disable interrupts**
- The running process will not be interrupted
- So, the critical section runs safely

➢ Problem:

- Works only for **uniprocessor**
- Not suitable for **multiprocessor systems**
- Not scalable

Hardware Support Methods

⇨ To handle critical sections efficiently, systems use the following:

1. Memory barriers
2. Hardware instructions
3. Atomic variables

Memory Barriers

⇨ A **memory model** defines how changes in memory made by one processor are seen by other processors.

- A **memory barrier** is a special instruction
- It forces all memory changes to be **visible to all processors**
- **Prevents** incorrect execution (**instruction reordering**) in multiprocessor systems

❖ There are two types of memory models:

1. Strongly Ordered Memory

- Any change in memory by one processor is **immediately visible** to all other processors

2. Weakly Ordered Memory

- Memory changes may **not be visible immediately**
- Other processors may see the change later

Example Scenario (Why Memory Barrier is Needed)

Shared Variable

```
boolean flag = false;
int x = 0;
```

Thread 1	Thread 2
`while (!flag);` `print(x);`	`x = 100;` `flag = true;`

➢ **Expected Output:** `100` (since `x` is set before `flag` becomes true)

❖ What Can Go Wrong?

⇨ Due to instruction reordering, Thread 2 may execute as:

Thread 2

```
flag = true;
x = 100;
```

➢ **Now:**

- Thread 1 sees `flag == true`
- Immediately prints `x`
- But `x` is still `0`

Output becomes `0` instead of `100`

❖ How Does a Memory Barrier Fix This?

Shared Variable

```
boolean flag = false;
int x = 0;
```

Thread 1	Thread 2

`while (!flag);`
`memory_barrier();`
`print(x);`
	`x = 100;`
	`memory_barrier();`
	`flag = true;`

➢ **What Happens Here?**

- Memory barrier forces `x = 100` to be visible
- Only then `flag` becomes true
- Thread 1 always sees correct value

Hardware Instructions

⇨ Some CPUs provide **special hardware instructions** that run **atomically** (cannot be interrupted also known as **atomic operation**). These instructions help implement **mutual exclusion** for critical sections.

They can:

- **Test and modify** a memory value
- Or **swap values** in one uninterruptible step

❖ **Two common hardware instructions are:**

1. **Test-and-Set**
2. **Compare-and-Swap**

Test-and-Set Instruction

➢ **What it Does**

- Executes **atomically**
- Returns the **old value** of a variable
- Sets the variable to **true**

❖ **Definition of Test-and-Set Instruction:**

```
boolean test_and_set(boolean *target) {
    boolean rv = *target;
    *target = true;
    return rv;
}
```

➢ **Meaning**

- Check the current value of `target`
- Set it to `true`
- Return the old value

Using Test-and-Set for Critical Section

Shared Variable

```
boolean lock = false;  // false = free, true = busy
```

1st Process 1	2nd Process 2
`do { while (test_and_set(&lock)); // busy waiting critical section lock = false; // release lock remainder section } while (true);`	`do { while (test_and_set(&lock)); // busy waiting critical section lock = false; // release lock remainder section } while (true);`

➢ **Meaning:**

- If `lock` is `false`, test-and-set sets it to `true` and enters critical section
- If `lock` is `true`, process keeps waiting
- Ensures **only one process** enters the critical section

Compare-and-Swap Instruction

➢ **What it Does**

- Executes **atomically**
- Compares a memory value with an expected value
- Updates the value **only if** they match
- Returns the **original value**

❖ **Definition of Compare-and-Swap Instruction:**

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

➢ **Meaning**

- If `*value == expected`, replace it with `new_value`
- Otherwise, do nothing
- Return the old value

Using Compare-and-Swap for Critical Section

Shared Variable

```
int lock = 0;   // 0 = free, 1 = busy
```

1st Process 1	2nd Process 2

`while (true) {`
`  while(compare_and_swap(&lock,0,1)!= 0);`
`  // busy waiting`
 	
`  critical section`
 	
`  lock = 0;  // release lock`
 	
`  remainder section`
`}`
	`while (true) {`
	`  while(compare_and_swap(&lock,0,1)!= 0);`
	`  // busy waiting`
	
	`  critical section`
	
	`  lock = 0;  // release lock`
	
	`  remainder section`
	`}`

➢ **Meaning:**

- If lock is `0`, it becomes `1` and process enters
- If lock is `1`, process keeps waiting
- Guarantees **mutual exclusion**

Bounded-Waiting Mutual Exclusion using Compare-and-Swap

⇨ This solution ensures:

- **Mutual Exclusion** → only one process enters critical section
- **Bounded Waiting** → no process waits forever (no starvation)
- Uses **compare-and-swap (CAS)** instruction

Shared Variable

```
int N = 3; // total number of processes
int lock = 0; // shared integer lock (0 = free, 1 = busy)
bool waiting[N] = {false,false,false}; // boolean array to track waiting processes
int i; // current process ID
```

1st Process 0

```
void process() {
    int key;
    int j;
    while (true) {
        /* ENTRY SECTION */
        waiting[i] = true;   // process i wants to enter
        key = 1;

        while (waiting[i] && key == 1) {
            key = compare_and_swap(&lock, 0, 1);
        }

        /* CRITICAL SECTION */

        waiting[i] = false;

        /* EXIT SECTION */
        j = (i + 1) % N; //Look for the next process in a circular manner

        while ((j != i) && waiting[j] == false) {
            j = (j + 1) % N;
        }
        if (j == i) {
            lock = 0;     // no one is waiting, release lock
        } else {
            waiting[j] = false; // give turn to next waiting process
        }
        /* REMAINDER SECTION */
    }
}
```

Atomic Variables

⇨ Variables that are updated **atomically**, meaning the update happens in **one uninterruptible step**.

- They are built using **atomic instructions** like `compare-and-swap.`
- Commonly used for **integers, booleans, counters**, etc.
- **Purpose:** Prevent race conditions in **multiprocessor systems**.

❖ Definition of Compare-and-Swap Instruction:

```
int compare_and_swap(atomic_int *value, int expected, int new_value) {
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

❖ Implementation of Atomic Variables Using Compare-and-Swap:

```
void increment(atomic_int *v) {
    int temp;
    do {
        temp = *v;  // read current value
    } while (temp != compare_and_swap(v, temp, temp + 1));
}
```

Shared Variable

```
typedef int atomic_int;
atomic_int sequence = 0;
```

1st thread 1	2nd thread 2
`increment(&sequence);`	`increment(&sequence);`

❖ **Note:** This increments the atomic variable `sequence` **safely**, without being interrupted by other threads.

Mutex Locks (Mutual Exclusion Locks)

- **Purpose:** Protect a **critical section** so that only **one process/thread** can enter at a time.
- **Used by:** Application programmers via OS-provided tools.
- **Simplest form:** A **boolean lock variable**.

How Mutex Works:

1. **Acquire the lock** before entering critical section
2. **Execute critical section**
3. **Release the lock** when done
4. **Do remainder work** outside critical section

Pseudo-Code Structure:

```
while (true) {
    acquire();          // Step 1: get lock
     // critical section
    release();          // Step 3: free lock
   // remainder section
}
```

Lock Functions:

<table>
<tr><th colspan="2">Shared Variable</th></tr>
<tr><td colspan="2">

```
available = true;
```

</td></tr>
<tr><th>Acquire Lock</th><th>Release Lock</th></tr>
<tr><td>

```
acquire() {
    while (!available) ;  // busy wait
    available = false;    // mark lock as taken
}
```

</td><td>

```
release() {
    available = true;    // mark lock as free
}
```

</td></tr>
<tr><td>

- **available** → boolean variable (`true` if lock is free)
- **busy wait:** keeps checking until lock becomes free
- **atomic:** the check + assignment must be **uninterruptible**

</td><td>

⇨ Allows **another thread** to acquire the lock

</td></tr>
</table>

Implementation Note

- `acquire()` and `release()` must be **atomic**.
- Typically implemented using **hardware instructions** like:
  - **Test-and-Set**
  - **Compare-and-Swap (CAS)**
- **Spinlock:** Because threads may **busy waiting**, this type of lock is also called a **spinlock**.
- **Busy Waiting:** **Busy waiting** is a situation where a process **keeps checking repeatedly** (spins in a loop) to see if a resource or condition becomes available **without giving up the CPU**.

Semaphore

⇨ **Semaphore** is a synchronization tool used to control access to shared resources by multiple processes or threads.

- More powerful than **Mutex locks**
- Used to **synchronize process activities**
- Represented by an **integer variable (S)**
- Can be accessed **only through atomic operations**

Atomic Operations on Semaphore

⇨ Semaphore supports **two indivisible (atomic) operations**:

❖ `wait()` **operation (also called** P()**)**

```
wait(S) {
    while (S <= 0);   // busy waiting
    S--;
}
```

- If `S ≤ 0`, the process **waits**
- When `S > 0`, it **decrements S and enters**

❖ `signal()` operation (also called **V()**):

```
signal(S) {
    S++;
}
```

- Increments the semaphore
- Wakes up a waiting process (if any)

Types of Semaphores

- **Binary Semaphore:**
  - Value ranges only between **0 and 1**
  - Acts like a **Mutex Lock**

- **Counting Semaphore:**
  - Value ranges from 0 **to** ∞
  - Used when **multiple resources** are available

Using Binary Semaphore for Critical Section

Shared Variable

```
semaphore S = 1;    // 1 = free; 0 = busy
```

1st Process 1

```
while (true) {
    wait(S);       // entry section

    /* critical section */

    signal(S);     // exit section

    /* remainder section */
}
```

2nd Process 2

```
while (true) {
    wait(S);       // entry section

    /* critical section */

    signal(S);     // exit section

    /* remainder section */
}
```

- It is used to ensure **mutual exclusion**, meaning **only one process can enter the critical section at a time**.

Working Explanation

1. Semaphore `S` is initialized to **1**
2. When a process executes `wait(S):`
   - If `S = 1`, it becomes `0` and the process enters the critical section
   - If `S = 0`, the process is **blocked**
3. While one process is inside the critical section, **no other process can enter**
4. After finishing, the process executes `signal(S)`
   - `S` becomes `1`
   - One waiting process (if any) is **woken up**

Using Counting Semaphore for Critical Section

❖ **Example: 3 Processes, 2 Resources:**

Shared Variable

```
semaphore S = 2;
```

1st Process 1

```
while (true) {
    wait(S);      // entry section

    /* critical section */

    signal(S);    // exit section

    /* remainder section */
}
```

2nd Process 2

```
while (true) {
    wait(S);      // entry section

    /* critical section */

    signal(S);    // exit section

    /* remainder section */
}
```

Step-by-Step Execution

Step	Process	S Value	Result
1	P1 calls wait	2 → 1	Enters CS
2	P2 calls wait	1 → 0	Enters CS
3	P3 calls wait	0	Blocked
4	P1 exits → signal	0 → 1	P3 wakes
5	P3 enters CS	1 → 0	Allowed

Using Semaphore for Ordering

Problem1: In a **printing system**, two concurrent processes are running:

- **Process P1** prepares a document.
- **Process P2** prints the document.

The printer must **not start printing until the document preparation is completed**.

a) Identify the synchronization problem in this system.

b) Using a **semaphore**, write a solution that ensures **document preparation happens before printing**.

c) Briefly explain how the semaphore enforces the correct execution order.

(a)

➔ The synchronization problem in this system is **execution order dependency**.

- **Process P2 (Printer)** depends on **Process P1 (Document Preparation)**
- Printing must not begin until the document is completely prepared
- If both processes run concurrently **without synchronization**, P2 may start printing before P1 finishes preparing the document. This leads to an **incorrect execution order** and a **race condition**.

(b)

Semaphore Initialization

```
semaphore S = 0;
S1 = Prepare the document
S2 = Print the document
```

Process P1 (Document Preparation)	Process P2 (Printer)

`S1;  // Prepare the document`
`signal(S);  // Notify that preparation is complete`
	`wait(S);  // Wait until document is ready`
	`S2;  // Print the document`

(c)

- The semaphore `S` is initialized to **0**
- When **P2 executes** `wait(S)`, it blocks because the semaphore value is 0
- **P1 executes S1**, completing document preparation
- P1 then calls `signal(S)`, increasing the semaphore value
- This signal **unblocks P2**
- P2 executes **S2 (printing)** only after S1 finishes

Thus, the semaphore guarantees that **S1 happens before S2**.

Semaphore Implementation Issue (Busy Waiting)

- `wait()` and `signal()` must be **atomic**
- They themselves become a **critical section**
- Basic implementation causes **busy waiting**
- Busy waiting wastes CPU time

Semaphore with NO Busy Waiting

⇨ Instead of spinning, the OS uses a **waiting queue**.

❖ **Semaphore Structure:**

```
typedef struct semaphore{
    int value;
    struct process *list;
} semaphore;
```

- `value` → semaphore count
- `list` → queue of blocked processes

❖ **Updated wait() & signal():**

Improved wait()

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}
```

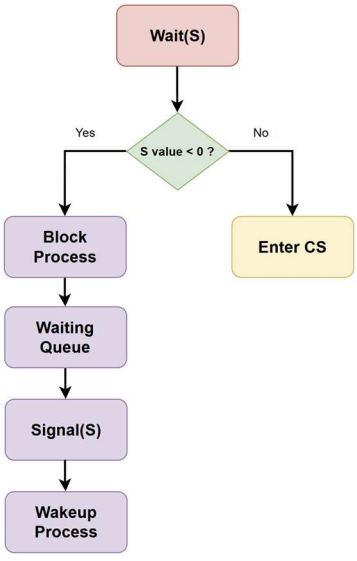
- If resource not available → process is **blocked**
- No CPU waste

Improved signal()

```
signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

- Increases resource count
- Wakes up **one waiting process**

Diagram (No Busy Waiting)



Problems with Semaphores

- Incorrect usage can cause **deadlock** or **starvation**.

❖ Common Mistakes:

- Calling `signal()` before `wait()`
- Multiple `wait()` without `signal()`
- Missing `wait()` or `signal()`
- Wrong ordering of semaphore operations

❖ **Example:**

```
signal(mutex);
wait(mutex);    // wrong order
```

Problem2: Two concurrent processes **P** and **Q** are synchronized using **two binary semaphores S1 and S2**, both initialized to **1**.
The processes execute the following code repeatedly:

Process P:	Process Q:

`P(S1);`
`P(S2);`
`Critical Section`
`V(S1);`
`V(S2);`
	`P(S2);`
	`P(S1);`
	`Critical Section`
	`V(S1);`
	`V(S2);`

a) Describe what happens during **normal execution** of the above scenario.
b) Describe what happens when **both processes attempt to execute at the same time**, that is, when **process P acquires semaphore S1 while process Q attempts to acquire semaphore S2 simultaneously**.
c) Does this solution satisfy the three requirements of the critical section problem (**Mutual Exclusion, Progress, and Bounded Waiting**)? Justify your answer, particularly when both processes attempt to execute at the same time.

(a)

➔ During normal execution (when the processes do **not interfere with each other**):

1. Suppose **Process P runs first**:
   - P executes `P(S1)` → S1 becomes 0, P has the lock on S1.
   - P executes `P(S2)` → S2 becomes 0, P has the lock on S2.
   - P enters the critical section and executes it safely.
   - After finishing, P executes `V(S1)` → S1 becomes 1.
   - P executes `V(S2)` → S2 becomes 1.
2. Now, **Process Q** can enter its critical section:
   - Q executes `P(S2)` → S2 becomes 0.
   - Q executes `P(S1)` → S1 becomes 0.
   - Q executes its critical section.
   - Q releases semaphores: `V(S1)` and `V(S2).`

So, If the processes execute one after another, there is **no problem**. Mutual exclusion is maintained.

(b)

➔ So, **P and Q start at the same time**:

- **Step 1:**
  - P executes `P(S1)` → S1 becomes 0.
  - Q executes `P(S2)` → S2 becomes 0.
- **Step 2:**
  - P now tries `P(S2)` → S2 is 0 → **P waits**.
  - Q now tries `P(S1)` → S1 is 0 → **Q waits**.

So, both processes are **waiting for the other process to release a semaphore**. This is a **classic deadlock**. Neither process can proceed. They are **stuck forever** unless an external intervention occurs.

(c)

Requirement	Status	Reason
Mutual Exclusion	Satisfied	Only one process can hold both semaphores.
Progress	Not satisfied	Deadlock may occur when both processes run simultaneously.
Bounded Waiting	Not satisfied	Processes may wait indefinitely in deadlock.

Deadlocks

⇨ A **deadlock** is a situation where a group of processes are **permanently blocked** because each process is holding a resource and waiting for another resource held by another process. No process can proceed.

System Model

- A system consists of **resources**
- Resource types:
  - CPU cycles
  - Memory
  - I/O devices

❖ Resource Instances

- Each resource type **Ri** has **Wi instances**
- A process uses resources in this order:

Deadlock in Multithreaded Applications

⇨ Deadlock can occur when **multiple threads** use **mutex locks** or **Binary Semaphore**

⇨ Example:

- Thread 1 locks `mutex1`
- Thread 2 locks `mutex2`
- Thread 1 waits for `mutex2`
- Thread 2 waits for `mutex1`

Thread 1:	Thread 2:

`P(mutex1);`
`P(mutex2);`
`Critical Section`
`V(mutex1);`
`V(mutex2);`
	`P(mutex2);`
	`P(mutex1);`
	`Critical Section`
	`V(mutex1);`
	`V(mutex2);`

❖ Result: **Deadlock**

Deadlock Characterization

⇨ Deadlock occurs **only if all four conditions hold at the same time**:

❖ Mutual Exclusion

- Only one process can use a resource at a time

❖ Hold and Wait

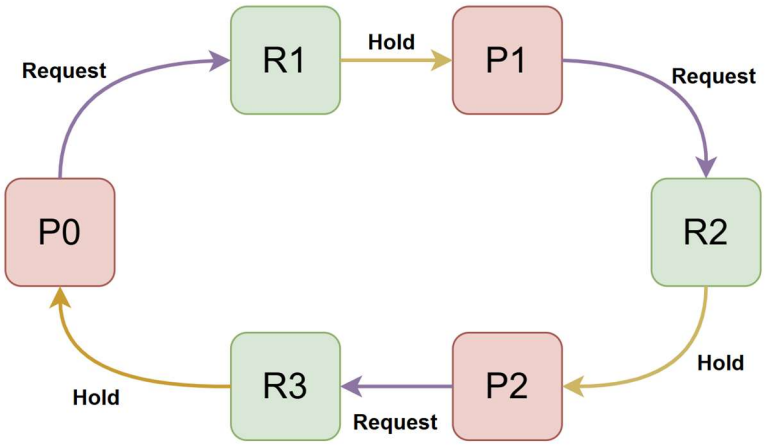
- A process holds one resource and waits for others

❖ No Preemption

- Resources cannot be forcibly taken away
- They must be released voluntarily

❖ Circular Wait

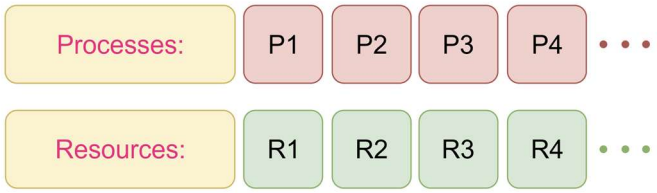
- Circular chain of processes waiting for resources



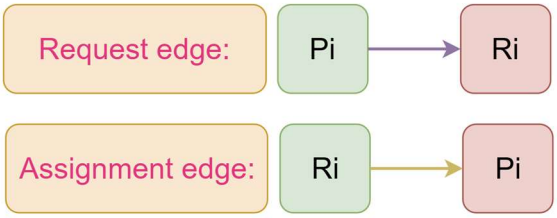
Resource Allocation Graph (RAG)

❖ Components

➢ Vertices (V):



➢ Edges (E):



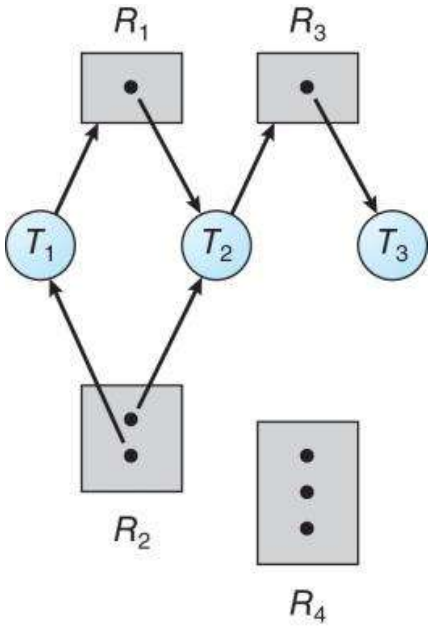
➢ Resource Allocation Scenario:

One instance of R1	One instance of R2
R1 	R2 
One instance of R3	**One instance of R4**
R3 	R4 

➢ Basic Facts:

- **No cycle → No deadlock**
- **Cycle + single instance → Deadlock**
- **Cycle + multiple instances → Possible deadlock**

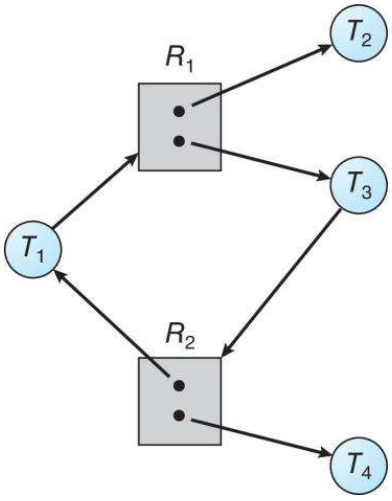
Resource Allocation Graph Example



- **T1**: already allocated / holds **R2** and requests **R1**
- **T2**: already allocated / holds **R1** and **R2** and requests **R3**
- **T3**: already allocated / holds **R3** and requests **nothing**

❖ **Note:** Since there is **no cycle** in the resource allocation graph**, no deadlock** occurs.

Graph With a Cycle but No Deadlock



Problem1: Consider a system with **three processes** and **three resource types**:

- **Processes**: P1, P2, P3
- **Resources**: R1, R2, R3 (each has **one instance**)

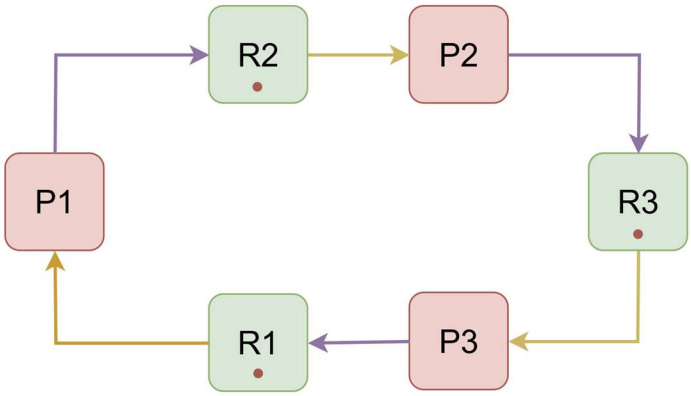
The current state of the system is as follows:

- **P1** holds **R1** and requests **R2**
- **P2** holds **R2** and requests **R3**
- **P3** holds **R3** and requests **R1**

a) Draw the **Resource Allocation Graph (RAG)**.

b) Does the system suffer from a **deadlock**? Explain your answer.

(a)



(b)

Is given:

- Total Number of Instances (TNI):
  - R1 = 1
  - R2 = 1
  - R3 = 1
- Formula:

$$\boldsymbol{Available = Total\ No.of\ Instances - Total\ Allocated}$$

- Rule for every resource:

$$\boldsymbol{If\ Request \leq Available \rightarrow request\ can\ be\ granted}$$

$$\boldsymbol{If\ Request > Available \rightarrow request\ cannot\ be\ granted}$$

Step1:

Processes	Allocated			Request			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
❌P1	1	0	0	0	1	0	0	0	0
P2	0	1	0	0	0	1			
P3	0	0	1	1	0	0			
Total	1	1	1						

Step2:

Processes	Allocated			Request			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
❌P1	1	0	0	0	1	0	0	0	0
❌P2	0	1	0	0	0	1	0	0	0
P3	0	0	1	1	0	0			
Total	1	1	1						

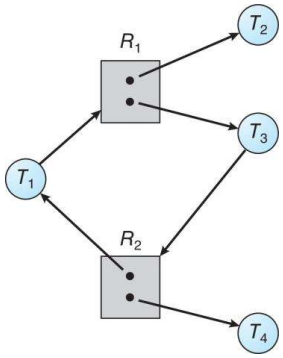
Step3:

Processes	Allocated			Request			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
❌P1	1	0	0	0	1	0	0	0	0
❌P2	0	1	0	0	0	1	0	0	0
❌P3	0	0	1	1	0	0	0	0	0
Total	1	1	1						

So, all processes are **waiting**, and none can proceed.

Since, all resources have only one instance and a cycle exist in the resource allocation graph, the system is in a **deadlock**.

Problem2: Consider the following Resource Allocation Graph (RAG):



a) Does the system suffer from a **deadlock**? Explain your answer.

(a)

Is given:

- Total Number of Instances (TNI):
  - R1 = 2
  - R2 = 2

Step1:

Processes	Allocated		Request		Available	
	R1	R2	R1	R2	R1	R2
✗ T1	0	1	1	0	0	0
T2	1	0	0	0		
T3	1	0	0	1		
T4	0	1	0	0		
Total	2	2				

Step2:

Processes	Allocated		Request		Available	
	R1	R2	R1	R2	R1	R2
✗ T1	0	1	1	0	0	0
✓ T2	0	0	0	0	0	0
T3	1	0	0	1	1	0
T4	0	1	0	0		
Total	1	2				

Step3:

Processes	Allocated		Request		Available	
	R1	R2	R1	R2	R1	R2
✗ T1	0	1	1	0	0	0
✗ T3	1	0	0	1	1	0
T4	0	1	0	0		
Total	1	2				

Step4:

Processes	Allocated		Request		Available	
	R1	R2	R1	R2	R1	R2
✗ T1	0	1	1	0	0	0
✗ T3	1	0	0	1	1	0
✓ T4	0	0	0	0	1	0
Total	1	1			1	1

Step5:

Processes	Allocated		Request		Available	
	R1	R2	R1	R2	R1	R2
✓ T1	0	0	1	0	0	0
✗ T3	1	0	0	1	1	0
Total	1	0			1	1
					1	2

Step6:

Processes	Allocated		Request		Available	
	R1	R2	R1	R2	R1	R2
✓ T3	0	0	0	1	1	0
Total	0	0			1	1
					1	2
					2	2

$$\textbf{\textit{Safe Sequence}}: T2 \rightarrow T4 \rightarrow T1 \rightarrow T3$$

Since a safe sequence **(T2, T4, T1, T3)** exists and all processes can complete their execution, the system **does not suffer from a deadlock**.

Methods for Handling Deadlocks

❖ Three Approaches:

1. **Deadlock Prevention**
2. **Deadlock Avoidance**
3. **Deadlock Detection & Recovery**

Deadlock Prevention

⇨ The Idea of Deadlock Prevention: **Break one of the four necessary conditions**

❖ Mutual Exclusion

- **Not required** for **sharable resources** (e.g., read-only files).

❖ Hold and Wait

⇨ To prevent deadlock, the system must ensure that:

- A process **does not hold any resource while requesting another**, or
- A process **requests and is allocated all required resources before execution begins**.

➢ **Disadvantages:**

- Low resource utilization
- Starvation may occur

❖ No Preemption

➢ If a process holding some resources requests another resource that **cannot be immediately allocated**:

- All currently held resources are **released (preempted)**.
- The released resources are added to the waiting list.

➢ The process is restarted **only when it can acquire all its old resources plus the new one**.

❖ Circular Wait

➢ Assign **each resource (mutex lock) a unique number**.

➢ **Threads must acquire resources in increasing order** of numbers.

➢ This **prevents circular wait**, and hence deadlock.

❖ **Example:**

Suppose we have:

```
first_mutex  = 1
second_mutex = 5
```

Correct way to acquire resources (thread_two):

```
lock(first_mutex);    // always acquire the lower-numbered mutex first
lock(second_mutex);   // then acquire the higher-numbered mutex

// critical section using both mutexes

unlock(second_mutex); // release in reverse order
unlock(first_mutex);
```

Deadlock Avoidance

1. **Requires a priori information**
   - The system must know in advance the **maximum number of resources that every process may need**.
2. **Maximum Claim Declaration**
   - Each process **declares its maximum resource needs** before execution.
   - This allows the system to **avoid unsafe allocations** dynamically.
3. **Resource-Allocation State**
   - Defined by:
     - **Available resources**
     - **Allocated resources**
     - **Maximum demands of each process**
4. **Deadlock-Avoidance Algorithm**
   - Examines the **current allocation state** to ensure that a resource will **not lead to a circular wait**.
   - Prevents unsafe allocations **before they happen**.

Safe State

- A system is in a **safe state** if **all processes can finish one by one** without getting stuck.
- There is a **safe sequence** of processes, like: < P1, P2, … , Pn >
  - **How it works:**
    1. Each process Pi **can get the resources that still needs** using:
       - **Resources currently available**
       - **Resources held by all processes that finished before it**
    2. If Pi cannot get all its needed resources, it **waits**.
    3. Once earlier processes finish, they **release their resources**.
    4. Then Pi can get its resources, run, and **release them**.
    5. Repeat this until **all processes are finished**.

  - If such a sequence exists, the system is **safe**.
  - If no such sequence exists, the system is **unsafe** (possible deadlock).

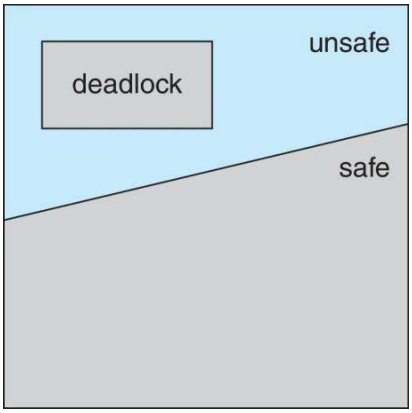
Basic Facts

1. **Safe state ⇒ no deadlocks**
   - If the system is in a safe state, all processes can finish safely.

2. **Unsafe state ⇒ possibility of deadlock**
   - Being in an unsafe state does not always mean deadlock has occurred, but there is a risk.

3. **Deadlock avoidance**
   - The system must ensure it **never enters an unsafe state**.

Safe, Unsafe, Deadlock State



Avoidance Algorithms

❖ Single instance of a resource type:

- Use a **Resource-Allocation Graph (RAG)**.

❖ Multiple instances of a resource type:

- Use the **Banker's Algorithm**.

Banker's Algorithm

❖ Required Data Structures:

Name	Description
Available	Free instances of each resource
Max Need	Maximum demand of each process
Allocation	Resources currently allocated
Remaining Need	Remaining resource need

➢ Formula:

$$Available = Total\ No.of\ Instances - Total\ Allocated$$

$$Remaining\ Need\ (RN) = Max\ Need - Each\ Allocation$$

➢ Rule for every resource:

If **RN** $\leq$ **Available** $\rightarrow$ request **can be granted**

If **RN** $>$ **Available** $\rightarrow$ request **cannot be granted**

Safety Algorithm

➢ Steps:

1. Work = Available
2. Finish[i] = false
3. Find process with:
   - Finish[i] = false
   - Need $\leq$ Work
4. Work = Work + Allocation
5. Mark Finish[i] = true
6. If all Finish = true $\rightarrow$ Safe State

Problem3: Consider a system with **four processes** and **three resource types**:

- **Processes**: P0, P1, P2, P3
- **Resource Types**:
  - E = 8 instances
  - F = 3 instances
  - G = 6 instances

The **current state of the system** is given in the table below:

Resource Allocation Table

Process	Allocation			Maximum		
	E	F	G	E	F	G
P0	1	0	1	4	3	1
P1	1	1	2	2	1	4
P2	1	0	3	1	3	3
P3	2	0	0	5	4	1

a) Calculate the **Remaining Need** for each process.
b) Does the system suffer from a **deadlock**? Explain your answer using the **safe-state (Banker's Algorithm) approach**.
c) What will happen if **resource F has 4 instances** instead of 3, and **no process has any allocation of resource F** in the allocation table?

(a)

We Know:

$$\textbf{\textit{Remaining Need (RM)} = \textit{Max Need} - \textit{Each Allocation}}$$

Process	Allocation			Maximum Need			Remaining Need		
	E	F	G	E	F	G	E	F	G
P0	1	0	1	4	3	1	3	3	0
P1	1	1	2	2	1	4	1	0	2
P2	1	0	3	1	3	3	0	3	0
P3	2	0	0	5	4	1	3	4	1

(b)

Is given:

- Total Number of Instances (TNI):
  - E = 8
  - F = 3
  - F = 6

$$Available = Total\ No.of\ Instances - Total\ Allocated$$

Process	Allocation			Maximum Need			Available			Remaining Need		
	E	F	G	E	F	G	E	F	G	E	F	G
× P0	1	0	1	4	3	1	3	2	0	3	3	0
× P1	1	1	2	2	1	4	3	2	0	1	0	2
× P2	1	0	3	1	3	3	3	2	0	0	3	0
× P3	2	0	0	5	4	1	3	2	0	3	4	1
Total	5	1	6									

Since, No process can proceed with the available resources,
And , No safe sequence exists; so, the system is **deadlocked**.

(c)

Is given:

- Total Number of Instances (TNI):
  - E = 8
  - F = 4
  - F = 6

⇨ **Step1:**

Process	Allocation			Maximum Need			Available			Remaining Need		
	E	F	G	E	F	G	E	F	G	E	F	G
✔ P0	1	0	1	4	3	1	3	4	0	3	3	0
P1	1	0	2	2	1	4				1	1	2
P2	1	0	3	1	3	3				0	3	0
P3	2	0	0	5	4	1				3	4	1
Total	5	0	6									

⇨ **Step2:**

Process	Allocation			Maximum Need			Available			Remaining Need		
	E	F	G	E	F	G	E	F	G	E	F	G
✔ P0	0	0	0	4	3	1	3	4	0	3	3	0
× P1	1	0	2	2	1	4	4	4	1	1	1	2
P2	1	0	3	1	3	3				0	3	0
P3	2	0	0	5	4	1				3	4	1
Total	4	0	5									

⇨ **Step3:**

Process	Allocation			Maximum Need			Available			Remaining Need		
	E	F	G	E	F	G	E	F	G	E	F	G
✔ P0	0	0	0	4	3	1	3	4	0	3	3	0
× P1	1	0	2	2	1	4	4	4	1	1	1	2
✔ P2	1	0	3	1	3	3	4	4	1	0	3	0
P3	2	0	0	5	4	1				3	4	1
Total	4	0	5									

⇨ **Step4:**

Process	Allocation			Maximum Need			Available			Remaining Need		
	E	F	G	E	F	G	E	F	G	E	F	G
✔ P0	0	0	0	4	3	1	3	4	0	3	3	0
× P1	1	0	2	2	1	4	4	4	1	1	1	2
✔ P2	0	0	0	1	3	3	4	4	1	0	3	0
✔ P3	2	0	0	5	4	1	5	4	4	3	4	1
Total	3	0	2									

⇨ **Step5:**

Process	Allocation			Maximum Need			Available			Remaining Need		
	E	F	G	E	F	G	E	F	G	E	F	G
✔ P0	0	0	0	4	3	1	3	4	0	3	3	0
✔ P1	1	0	2	2	1	4	4	4	1	1	1	2
✔ P2	0	0	0	1	3	3	4	4	1	0	3	0
✔ P3	0	0	0	5	4	1	5	4	4	3	4	1
Total	1	0	2				7	4	4			

⇨ **Step6:**

Process	Allocation			Maximum Need			Available			Remaining Need		
	E	F	G	E	F	G	E	F	G	E	F	G
✔ P0	0	0	0	4	3	1	3	4	0	3	3	0
✔ P1	0	0	0	2	1	4	4	4	1	1	1	2
✔ P2	0	0	0	1	3	3	4	4	1	0	3	0
✔ P3	0	0	0	5	4	1	5	4	4	3	4	1
Total	0	0	0				8	4	6			

$$\textbf{\textit{Safe Sequence}}: \boldsymbol{P0 \rightarrow P2 \rightarrow P3 \rightarrow P1}$$

Since a safe sequence exists and all processes can complete their execution,
So, the system **does not suffer from a deadlock**.

Problem4: Consider a system with **four processes** and **three resource types**:

- **Processes**: P0, P1, P2, P3, P4
- **Resource Types**:
  - A = 10 instances
  - B = 5 instances
  - C = 7 instances

The **current state of the system** is given in the table below:

Resource Allocation Table

Process	Allocation			Maximum		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

a) Calculate the **Remaining Need** for each process.
b) Does the system suffer from a **deadlock**? Explain your answer using the **safe-state (Banker's Algorithm) approach**.

(a)

We Know:

$$\textbf{\textit{Remaining Need (RM)} = \textit{Max Need} - \textit{Each Allocation}}$$

Process	Allocation			Maximum Need			Remaining Need		
	A	B	C	A	B	C	E	F	G
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

(b)

Is given:

- Total Number of Instances (TNI):
  - A = 10
  - B = 5
  - C = 7

$$\textbf{\textit{Available} = \textit{Total No. of Instances} - \textit{Total Allocated}}$$

Process	Allocation			Maximum Need			Available			Remaining Need		
	A	B	C	A	B	C	A	B	C	A	B	C
× P0 4th	0	$1_{(0)}$	0	7	5	3	3	3	2	7	4	3
✔ P1 1st	$2_{(0)}$	0	0	3	2	2	3	3	2	1	2	2
× P2 5th	$3_{(0)}$	0	$2_{(0)}$	9	0	2	5	3	2	6	0	0
✔ P3 2nd	$2_{(0)}$	$1_{(0)}$	$1_{(0)}$	2	2	2	5	3	2	0	1	1
✔ P4 3rd	0	0	$2_{(0)}$	4	3	3	7	4	3	4	3	1
Total	7	2	5	✔ P0 4th			7	4	5	7	4	3
After P1	5	2	5	✔ P2 5th			7	5	5	6	0	0
After P3	3	1	4									
After P4	3	1	2									
After P0	3	0	2									

$$\textbf{\textit{Safe Sequence}: P1} \rightarrow \textbf{\textit{P3}} \rightarrow \textbf{\textit{P4}} \rightarrow \textbf{\textit{P0}} \rightarrow \textbf{\textit{P2}}$$

Since a safe sequence exists and all processes can complete their execution,
So, the system **does not suffer from a deadlock**.

Or

Is given:

- Total Number of Instances (TNI):
  - A = 10
  - B = 5
  - C = 7

Process	Allocation			Maximum Need			Remaining Need		
	A	B	C	A	B	C	A	B	C
P0 4th	0	1	0	7	5	3	7	4	3
P1 1st	2	0	0	3	2	2	1	2	2
P2 5th	3	0	2	9	0	2	6	0	0
P3 2nd	2	1	1	2	2	2	0	1	1
P4 3rd	0	0	2	4	3	3	4	3	1
Total	7	2	5						

Available				Completed process
Steps	A	B	C	
1	3	3	2	P1
2	3+2=5	3	2	P3
3	5+2=7	3+1=4	2+1=3	P4
4	7	4	3+2=5	P0
5	7	4+1=5	5	P2
6	7+3=10	5	5+2=7	All process executed safely

$$\boldsymbol{Safe\ Sequence: P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2}$$

Since a safe sequence exists and all processes can complete their execution,
So, the system **does not suffer from a deadlock**.

Resource-Request Algorithm

⇨ To decide **whether a process can safely get requested resources** without causing **deadlock**.

⇨ A request is **granted immediately only if** all of these are true:

❖ **Steps:**

1. **Check Need**:

If $\text{Request}_i \leq \text{Need}_i \rightarrow$ Continue

Else → Error (exceeds maximum claim)

2. **Check Availability**:

If $\text{Request}_i \leq \text{Available} \rightarrow$ Continue

Else → $Pi\ waits$

3. **Check Safety after allocation** (Safety check passes → system remains in safe state)

$$New\ Available = Available - Request_i$$
$$New\ Allocation_i = Allocation_i + Request_i$$
$$New\ Need_i = Need_i - Request_i$$

Problem5: Ans the following questions using Banker's Algorithm:

Process	Allocation				Max				Available			
	A	B	C	D	A	B	C	D	A	B	C	D
P1	1	2	3	5	7	8	5	9	1	1	2	2
P2	3	4	3	1	8	7	4	3				
P3	2	2	4	4	3	2	4	4				
P4	2	2	3	2	8	6	5	5				
P5	3	3	4	2	5	5	7	3				

a) Find the **Need Matrix**.

b) Determine whether the system is in a **safe state** or not.

c) If process **P2** requests **(1, 2, 2, 1)**, can the request be granted immediately? if yes will it be in a safe state?

(a)

We Know:

$$\textbf{\textit{Remaining Need (RM)} = \textit{Max Need} - \textit{Each Allocation}}$$

Process	Allocation				Max				Need			
	A	B	C	D	A	B	C	D	A	B	C	D
P1	1	2	3	5	7	8	5	9	6	6	2	4
P2	3	4	3	1	8	7	4	3	5	3	1	2
P3	2	2	4	4	3	2	4	4	1	0	0	0
P4	2	2	3	2	8	6	5	5	6	4	2	3
P5	3	3	4	2	5	5	7	3	2	2	3	1

(b)

Is given:

Available					Completed process
Steps	A	B	C	D	
1	1	1	2	2	P3
2	1+2=3	1+2=3	4+2=6	4+2=6	P5
3	3+3=6	3+3=6	6+4=10	6+2=8	P1
4	6+1=7	6+2=8	10+3=13	8+5=13	P2
5	7+3=10	8+4=12	13+3=16	13+1=14	P4
6	10+2=12	12+2=14	16+3=19	14+2=16	All process executed safely

$$\textbf{\textit{Safe Sequence}: P3} \rightarrow \textbf{P5} \rightarrow \textbf{P1} \rightarrow \textbf{P2} \rightarrow \textbf{P4}$$

Since a safe sequence exists and all processes can complete their execution,
So, the system **does not suffer from a deadlock**.

(c)

- **For P2 requests (1, 2, 2, 1):**
  - **Check Need:**

$$\textbf{Request}_2 \leq \textbf{Need}_2 = (1, 2, 2, 1) <\neq (5, 3, 1, 2)$$

Note: Request cannot be granted immediately. **Safety check not required**

Problem6:

Process	Allocation			Maximum			Available		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	2	3	1
P1	2	0	0	3	2	2			
P2	3	0	2	9	0	2			
P3	2	1	1	2	2	2			
P4	0	0	2	4	3	3			

a) Find the **Need Matrix**.
b) Determine whether the system is in a **safe state** or not.
c) If process **P0** requests **(0, 2, 0)** can the request be granted immediately? if yes will it be in a safe state?
d) If process **P4** requests **(3, 3, 1)** can the request be granted immediately? if yes will it be in a safe state?

(a)

We Know:

$$\textbf{\textit{Remaining Need (RM)} = \textit{Max Need} - \textit{Each Allocation}}$$

Process	Allocation			Maximum Need			Remaining Need		
	A	B	C	A	B	C	E	F	G
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

(b)

Here:

	Available			Completed process
Steps	A	B	C	
1	2	3	1	P3
2	4	4	2	P4
3	4	4	4	P1
4	6	4	4	P2
5	9	4	6	P0
6	9	5	6	All process executed safely

$$\textbf{\textit{Safe Sequence}: P3} \rightarrow \textbf{\textit{P4}} \rightarrow \textbf{\textit{P1}} \rightarrow \textbf{\textit{P2}} \rightarrow \textbf{\textit{P0}}$$

Since a safe sequence exists and all processes can complete their execution,
So, the system **does not suffer from a deadlock**.

(c)

❖ **For P0 requests (0, 2, 0):**

➢ **Check Need:**

$$\textbf{Request}_0 \leq \textbf{Need}_0 = (0,2,0) \leq (7,4,3)$$

➢ **Check Availability:**

$$\textbf{Request}_0 \leq \textit{Available} = (0,2,0) \leq (2,3,1)$$

$$\therefore \textit{New Available} = \textit{Available} - \textit{Request}_0 = (2,3,1) - (0,2,0) = (2,1,1)$$
$$\therefore \textit{New Allocation}_0 = \textit{Allocation}_0 + \textit{Request}_0 = (0,1,0) + (0,2,0) = (0,3,0)$$
$$\therefore \textit{New Need}_0 = \textit{Need}_0 - \textit{Request}_0 = (7,4,3) - (0,2,0) = (7,2,3)$$

➢ **Check Safety after allocation:**

❖ Updated Allocation & Need Table

Process	Allocation			Maximum Need			Remaining Need		
	A	B	C	A	B	C	E	F	G
P0	0	3	0	7	5	3	7	2	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

❖ Updated Available Table

	Available			Completed process
Steps	A	B	C	
1	2	1	1	P3
2	4	2	2	P1
3	6	2	2	P2
4	9	2	4	P0
5	9	5	4	P4
6	9	5	6	All process executed safely

$$\textbf{\textit{Safe Sequence}}: \textbf{P3} \rightarrow \textbf{P1} \rightarrow \textbf{P2} \rightarrow \textbf{P0} \rightarrow \textbf{P4}$$

Since a safe sequence is exists so, process **P0** requests **(0, 2, 0)** can granted immediately & it will be in a safe state?

(d)

❖ **For P4 requests (3, 3, 0):**

➢ **Check Need:**

$$\text{Request}_4 \leq \text{Need}_4 = (3,3,1) \leq (4,3,1)$$

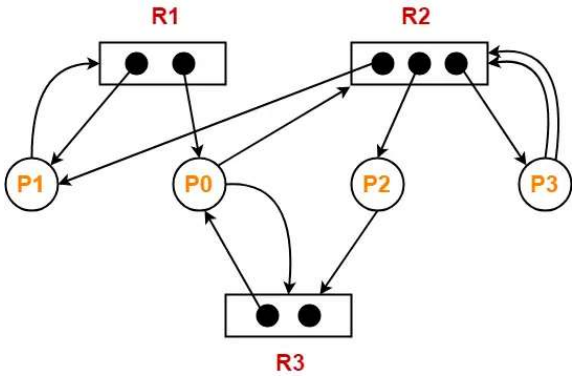
➢ **Check Availability:**

$$\text{Request}_4 \leq \textit{Available} = (3,3,1) <\neq (2,3,1)$$

Since, $\text{Request}_4 > \textit{Available}$

So, Request cannot be granted immediately.

Problem7: Consider the following Resource Allocation Graph (RAG):



a) Does the system suffer from a **deadlock**? If no, then find the find a safe sequence.

➢ **Solution:**

➢ Total Number of Instances (TNI):

- R1 = 2
- R2 = 3
- R3 = 2

Processes	Allocated			Request		
	R1	R2	R3	R1	R2	R3
P0	1	0	1	0	1	1
P1	1	1	0	1	0	0
P2	0	1	0	0	0	1
P3	0	1	0	0	2	0
Total	2	3	1			

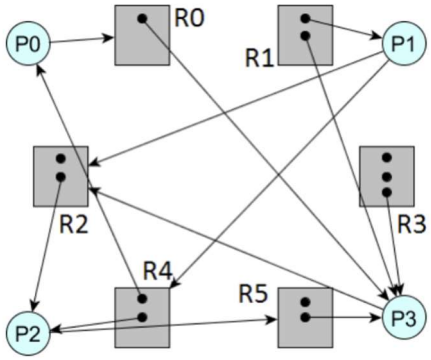
➢ **Available Table:**

Available				Completed process
Steps	R1	R2	R3	
1	0	0	1	P2
2	0	1	1	P0
3	1	1	2	P1
4	2	2	2	P3
5	2	3	0	All process executed safely

$$\boldsymbol{Safe\ Sequence: P2 \rightarrow P0 \rightarrow P1 \rightarrow P3}$$

Since a safe sequence exists and all processes can complete their execution,
So, the system **does not suffer from a deadlock**.

Problem8: Consider the following Resource Allocation Graph (RAG):



a) Is there a deadlock in this Resource Allocation Graph (RAG)?
b) If a deadlock exists, how can it be **prevented**?

(a)

➢ Total Number of Instances (TNI):

- R0 = 1
- R1 = 2
- R2 = 2
- R3 = 3
- R4 = 2
- R5 = 2

Processes	Allocated						Request					
	R0	R1	R2	R3	R4	R5	R0	R1	R2	R3	R4	R5
P0	0	0	0	0	1	0	1	0	0	0	0	0
P1	0	1	0	0	0	0	0	0	1	0	1	0
P2	0	0	1	0	1	0	0	0	0	0	0	1
P3	1	1	0	1	0	1	0	0	1	0	0	0
Total	1	2	1	1	2	1						

➢ **Available Table:**

Available							Completed process
Steps	R0	R1	R2	R3	R4	R5	
1	0	0	1	2	0	1	P2
2	0	0	2	2	1	1	P3
3	1	1	2	3	1	2	P0
4	1	1	2	3	2	2	P1
5	1	2	2	3	2	2	All process executed safely

$$\textit{Safe Sequence}: P2 \rightarrow P3 \rightarrow P0 \rightarrow P1$$

Since a safe sequence exists and all processes can complete their execution,
So, the system **does not suffer from a deadlock**.

(b)

⇨ The Idea of Deadlock Prevention: **Break one of the four necessary conditions**

❖ Mutual Exclusion

- **Not required** for **sharable resources** (e.g., read-only files).

❖ Hold and Wait

⇨ To prevent deadlock, the system must ensure that:

- A process **does not hold any resource while requesting another**, or
- A process **requests and is allocated all required resources before execution begins**.

➢ **Disadvantages:**

- Low resource utilization
- Starvation may occur

❖ No Preemption

➢ If a process holding some resources requests another resource that **cannot be immediately allocated**:

- All currently held resources are **released (preempted)**.
- The released resources are added to the waiting list.

➢ The process is restarted **only when it can acquire all its old resources plus the new one**.

❖ Circular Wait

➢ Assign **each resource (mutex lock) a unique number**.
➢ **Threads must acquire resources in increasing order** of numbers.
➢ This **prevents circular wait**, and hence deadlock.

MEMORY MANAGEMENT

Memory & CPU

- A **program must be loaded from disk into main memory** before the CPU can run it.
- The **CPU can directly access only registers and main memory**.
- The **memory unit does not understand programs**; it only handles:
  - address + read request
  - address + data + write request
- **Registers are very fast** and can be accessed in **one CPU clock cycle**.
- **Main memory is slow** and may take **many CPU cycles**, causing the CPU to **stall (wait)**.
- **Cache memory** is placed **between CPU registers and main memory** to reduce delay.
- **Memory protection is necessary** to prevent one process from accessing another process's memory and to ensure correct system operation.

Memory Protection

⇨ A process must be allowed to **access only the memory addresses that belong to it**.

⇨ This is called **memory protection**.

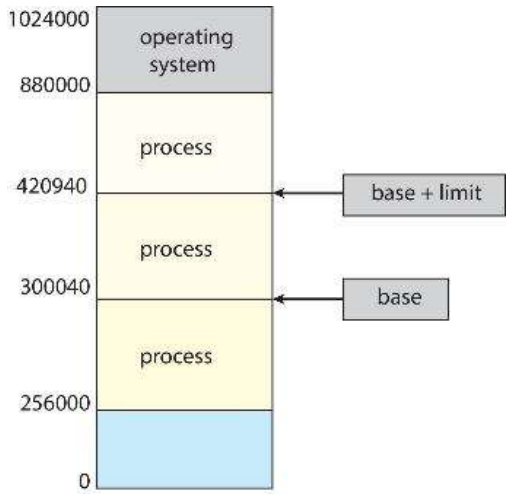
⇨ Memory protection is implemented using **base and limit registers**.

⇨ The **base register** stores the **starting address** of the process in memory.

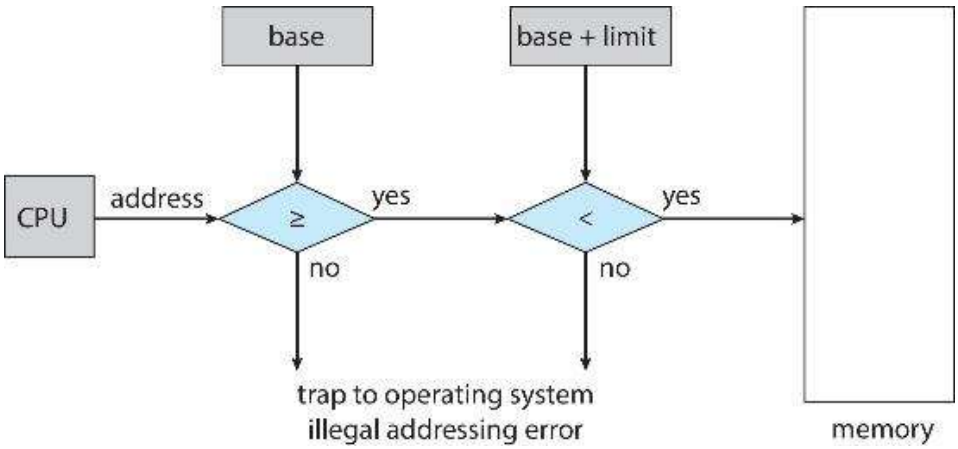
⇨ The **limit register** stores the **size (range)** of the process's address space.

⇨ The CPU checks every memory access:

- If the address is **within base and limit → access allowed**
- If the address is **outside the range → access denied**



Hardware Address Protection



- ⇨ The **CPU checks every memory access** made by a process in **user mode**.
- ⇨ It ensures that the accessed address is **between the base and limit registers**.
- ⇨ If the address is **within the allowed range**, access is granted.
- ⇨ If the address is **outside the range**, a **protection fault** occurs.
- ⇨ The **instructions to load (set) the base and limit registers are privileged**.
- ⇨ Only the **operating system (kernel mode)** can change base and limit values.
- ⇨ This prevents user programs from accessing **unauthorized memory**.

Address Binding

- ⇨ Programs stored on disk and ready to run are kept in an **input queue**.
- ⇨ When a program is loaded into memory, it must be placed at some **memory address**.
- ⇨ Without special support, a program would always have to be loaded at **physical address 0000**, which is **not practical**.
- ⇨ To solve this, **addresses are represented differently at different stages** of a program's life.

❖ Types of Addresses

1. **Symbolic Address (Source Code)**
   - Used in source code
   - Example: variable names like `count`, `total`
2. **Relocatable Address (Compiled Code)**
   - Generated by the compiler
   - Example: **"14 bytes from the beginning of this module"**
3. **Absolute Address (Execution Time)**
   - Generated by linker or loader
   - Example: **74014**
4. **Linker or loader** converts relocatable addresses into absolute addresses.

5. This process is called **address binding**.
6. Address binding lets a program run **from any memory location**, not just 0000.

Binding of Instructions and Data to Memory

⇨ **Compile time:** Absolute addresses generated; recompile needed if start address changes.
⇨ **Load time:** Relocatable code; loader assigns addresses when loading.
⇨ **Execution time:** Addresses bound during run; process can move in memory; needs hardware support (base & limit registers).

Dynamic Loading

- The **entire program does not need to be in memory** at once.
- A **routine is loaded only when it is called**.
- **Unused routines are never loaded**, saving memory.
- All routines stay on **disk in relocatable format**.
- Useful for **large programs** with **rarely used code**.
- **No special OS support required**.
- Implemented by **program design**, with optional OS libraries.

Logical vs. Physical Address Space

- **Logical address (virtual address):** Generated by the **CPU**.
- **Physical address:** Actual address used by the **memory unit**.
- In **compile-time and load-time binding**, logical and physical addresses are **the same**.
- In **execution-time binding**, logical and physical addresses are **different**.
- **Logical address space:** All logical addresses generated by a program.
- **Physical address space:** All physical addresses used in memory.
- Separating logical and physical address spaces is **essential for memory management**.

Swapping

❖ What is Swapping?

⇨ **Swapping** is a technique where a **process is temporarily moved from main memory (RAM) to disk**.
⇨ Later, the process is **bring back into memory** to continue execution.
⇨ This allows the **total memory used by processes to be more than the actual physical RAM**.

❖ Backing Store:

- A **backing store** is a **fast disk** used to store swapped-out processes.
- It must:
  - Be **large enough** to store memory images of all processes
  - Provide **direct access** to these memory images

❖ Roll Out, Roll In:

- Used in **priority-based scheduling**.
- **Roll out**: A **low-priority process** is swapped out of memory.
- **Roll in**: A **high-priority process** is loaded (swapped in) into memory and executed.

❖ Swap Time:

- The **major part of swap time** is the **data transfer time** between main memory and disk.
- **Swap time ∝ amount of memory swapped**
  - More memory → more time

❖ Ready Queue:

- The system maintains a **ready queue**:
  - Contains **ready-to-run processes**
  - Their **memory images are stored on disk**

❖ Physical Address Issue:

⇨ **Does a swapped-out process return to the same physical address?**

➢ **Depends on address binding method:**

- If binding is done at compile or load time → same address needed
- If binding is done at execution time → any free address can be used

❖ I/O Consideration:

- Swapping must consider **pending I/O operations**.
- A process **should not be swapped out** if I/O is directly accessing its memory.

Swapping in Modern Operating Systems

⇨ Modified versions of swapping exist in:

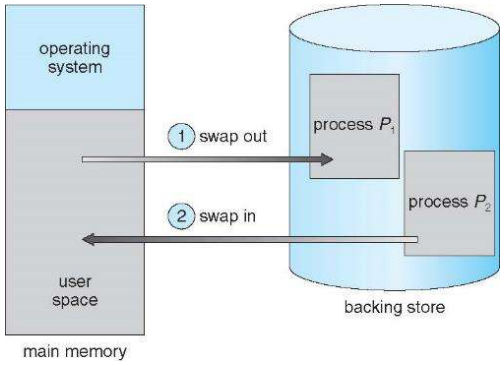
- **UNIX**
- **Linux**
- **Windows**

⇨ Full swapping is **normally disabled**.

When Swapping is Enabled or Disabled

- ⇨ **Enabled**: When memory usage **exceeds a threshold**
- ⇨ **Disabled**: When memory demand **falls below the threshold**

Schematic View of Swapping



Context Switch with Swapping

- ⇨ If the **next process to be put on CPU is not in main memory**, the OS must:
  - **Swap out** a process from memory to disk
  - **Swap in** the required process from disk to memory
- ⇨ This makes the **context switch time very high**.

❖ Example:

- Process size = **100 MB**
- Disk transfer rate = **50 MB/sec**

- ➢ **Swap-out time**: 100 MB ÷ 50 MB/sec = **2 seconds (2000 ms)**
- ➢ **Swap-in time**: Same size → **2 seconds**
- ➢ **Total swapping time in context switch**: 2 sec (out) + 2 sec (in) = **4 seconds (4000 ms)**

Very slow compared to normal context switching

❖ Reducing Swapping Time:

- Reduce the **amount of memory swapped**
- OS can swap **only the memory actually being used**
- Processes can inform the OS using:
  - `request_memory()` → request required memory
  - `release_memory()` → release unused memory

❖ Other Constraints on Swapping:

1. Pending I/O

- A process **cannot be swapped out** if: It has **pending I/O operations**
- Reason: I/O may write data into **wrong memory space**

2. Double Buffering

- Solution: Transfer I/O data **first to kernel space**, then to device
- This is called **double buffering**
- Drawback: **Extra overhead**

Swapping in Modern Operating Systems

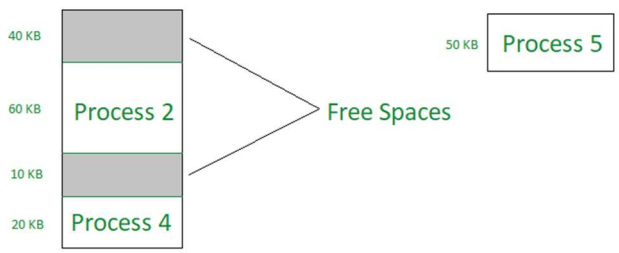
⇨ **Standard swapping** is **not used** today
⇨ **Modified swapping** is common
⇨ Swapping occurs **only when free memory is extremely low.**

Fragmentation

⇨ **Fragmentation** occurs when memory is wasted due to how it is allocated.

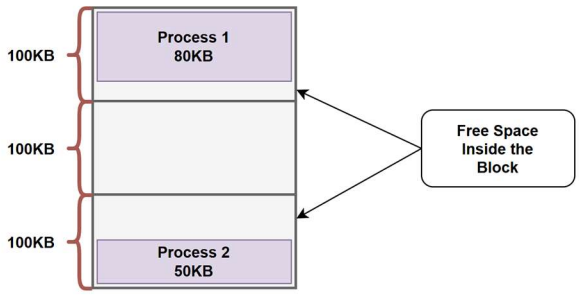
❖ External Fragmentation:

- **What it is:** Enough total free memory exists, **but not in one continuous block**.
- **Problem:** A process cannot fit even though total free memory is enough.
- **Solution: Compaction** – shuffle memory to combine free spaces into one big block.



❖ Internal Fragmentation:

- Allocated memory block is **slightly bigger than requested**, leaving unused space inside.
- **Example:** Request 28 KB, block allocated = 32 KB → 4 KB wasted.



❖ 50% Rule (First-Fit Analysis):

- Applies to **First-Fit memory allocation**.
- If **N memory blocks** are allocated: About **0.5 N blocks** are **lost due to fragmentation**.
- Out of total memory: Around **1/3 of memory becomes unusable**.
- This happens because:
  - First-fit creates **many small free holes**.
  - These holes are **too small to allocate** new processes.
- Type of fragmentation: **External fragmentation**.

Reducing External Fragmentation – Compaction

⇨ **Compaction:** Move (shuffle) processes in memory so that **all free space comes together** as **one large block**.

⇨ **When possible:**

- Only works if **dynamic relocation** is used.
- Done **during execution time**.

⇨ **I/O problem:**

- A process doing I/O **cannot be moved**.
- Solution:
  - **Lock (latch)** the job in memory during I/O, or
  - Do **I/O through OS buffers** instead of process memory.

⇨ **Limitation:** Even backing store (disk) can suffer from **fragmentation**.

VI EDITOR

vi Editor

❖ What is vi Editor?

- **vi** stands for **Visual Editor**
- It is a **text editor** used in **Unix/Linux**
- Installed by **default** in almost all Unix systems
- Very **powerful and fast**
- **vim** is an improved version of vi
- Mostly **vi editor** is available in Linux systems

❖ Modes of vi Editor

⇨ vi editor works in **two modes**:

Command Mode:	Insert Mode:

• Default mode when vi starts
• Used to:
  ○ Move cursor
  ○ Delete text
  ○ Save and quit etc.
• Typed keys work as **commands**
• You must be in **command mode** to give commands
	• Used to **type or insert text**
	• Text you type appears in the file
	• Press **Esc** to return to command mode

❖ Switching Between Modes

- vi starts in **command mode**
- Press `i` → enters **insert mode**
- Type text normally
- Press `Esc` → goes back to **command mode**

Note:

- vi is **case-sensitive**
- Commands like `x`, `dd`, `i` must be typed correctly

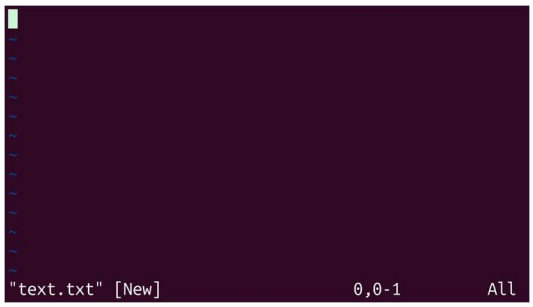
Creating or Opening a File

⇨ Opens an existing file.

⇨ Creates a new file if it does not exist

Command:	Example:
`vi filename`	`vi text.txt`

Output:



Saving and Quitting `vi`

⇨ Make sure you are in **command mode**.

❖ **Save file:** To **save the current file without quitting**

Command:	Output:
`:w`	

❖ **Quit in vi Editor:** To **quit vi** (only if no changes were made)

Command:	Output:

`:q`	`mohsin@X1-Yoga:~/final$ vi text.txt`
	`mohsin@X1-Yoga:~/final$ ls`
	`text.txt`
	`mohsin@X1-Yoga:~/final$`

Note: After saving and quitting the file in vi, if we run the `ls` command, we can see that the file now exists in the directory.

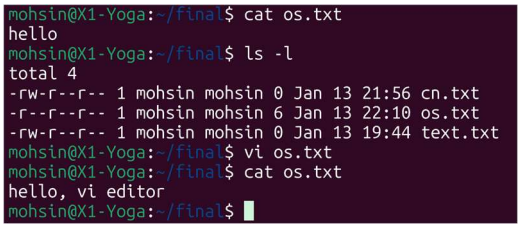
❖ **Save and quit:** used to save the file and exit `vi`

Command:	Output:
`:wq`	
Note: After using `:wq`, if we run the `ls` command, the file will be visible in the directory.	

❖ **Quit without saving:** used to exit vi without saving any changes.

Command:	Output:
`:q!`	
Note: Any modifications made after the last save will be **lost**.	

❖ **Force Save:** To **save the file forcibly**, even if it is **read-only**

Command:	Output:
`:w!`	

❖ Save as a **new file** with a different name**:**

Command:	Output:
`:w newfilename`	
Note: The original file remains unchanged; a **new file** is created with the given name.	

vi Commands to Start Typing

❖ Insert before cursor:

Command:	Output:
`i`	

Note: Press **Esc** to return to **Command Mode** after inserting. & To understand the output, **track the cursor position.**

❖ Insert at beginning of line:

Command:	Output:
`I`	

❖ Insert after cursor:

Command:	Output:
`a`	

❖ Insert at end of line:

Command:	Output:
`A`	

❖ New line below:

Command:	Output:
o	

❖ New line above:

Command:	Output:
O	

Note: To understand the output, **track the cursor position.**

vi Commands to Move Around a File

❖ Left Movement:

Command:	Output: starting index was 16.
h	
Note: It is used to move the cursor **one position to the left.**	

❖ Right Movement:

Command:	Output: starting index was 15.
l	
Note: It is used to move the cursor **one position to the right.**	

❖ Up Movement:

Command:	**Output:** Initially, cursor was on line 2.
k	
Note: It is used to move the cursor **one line up.**	

❖ Down Movement:

Command:	**Output:** Initially, cursor was on line 2.
j	
Note: It is used to move the cursor **one line down.**	

❖ Start of line:

Command:	**Output:** Initially, cursor was on index 8.
0	
Note: It is used to move the cursor to the **beginning of the current line.**	

❖ End of line:

Command:	**Output:** Initially, cursor was on index 1.
$	
Note: It is used to move the cursor to the **end of the current line.**	

❖ First line:

Command:	Output: Initially, cursor was on line 3.
gg	
Note: It is used to move the cursor to the **first line of the file.**	

❖ Last line:

Command:	Output: Initially, cursor was on line 1.
G	
Note: It is used to move the cursor to the **last line of the file.**	

vi Commands to Delete

❖ Delete character:

Command:	Output: Initially, cursor was on char "h".
x	
Note: It is used to delete **the character under the cursor**	

❖ Delete whole line:

Command:	Output: Initially, cursor was on line 3.
dd	
Note: It is used to **delete the entire current line.**	

❖ Delete word:

Command:	**Output:** Initially, cursor was on word “hello”.
dw	
Note: It is used to **delete the word under the cursor.**	

❖ Delete from cursor to end of line:

Command:	**Output:** Initially, cursor was on char “i”.
D	
Note: It is used to **delete everything from the cursor position to the end of the current line.**	

❖ Delete the character before the cursor:

Command:	**Output:** Initially, cursor was on char “d”.
X	

❖ Replace the character under the cursor:

Command:	**Output:** Initially, cursor was on char “d”.
r	
Note: After pressing `r`, type **one character** to replace the current one.	

❖ **switch (swap) the current character with the previous one:**

Command:	**Output:** Initially, cursor was on char "A".
`xp`	

How it works:

- `x` → deletes the character under the cursor
- `p` → pastes it **after** the cursor
- Result: the two characters are **swapped**

Cut, Copy, and Paste Commands

❖ **Copy (yank) line:**

Command:	**Output:** Initially, cursor was on line 1.
`yy`	

Note: It is used to **copy (yank) the current line.**

❖ **Paste after cursor:**

Command:	**Output:** Initially, cursor was on line 1.
`p`	

Note: It is used to **paste the copied or deleted text after the cursor.**

❖ Paste before cursor:

Command:	**Output:** Initially, cursor was on line 2.
P	
Note: It is used to **paste the copied or deleted text before the cursor.**	

❖ Cut line:

Command:	**Output:** Initially, cursor was on line 1.
dd	
Note: It is used to **cut (remove) the current line.**	

Saving and Quitting vi

Command	Action
`:w`	Save file
`:q`	Quit
`:wq`	Save and quit
`:q!`	Quit without saving

vi Commands to Start Typing

Command	Function
`i`	Insert before cursor
`I`	Insert at beginning of line
`a`	Insert after cursor
`A`	Insert at end of line
`o`	New line below
`O`	New line above

vi Commands to Move Around a File

Key	Movement
`h`	Left
`l`	Right
`k`	Up
`j`	Down
`0`	Start of line
`$`	End of line
`gg`	First line
`G`	Last line

vi Commands to Delete

Command	Action
`x`	Delete character
`dd`	Delete whole line
`dw`	Delete word
`D`	Delete from cursor to end of line

Cut, Copy, and Paste Commands

Command	Action
`yy`	Copy (yank) line
`dd`	Cut line
`p`	Paste after cursor
`P`	Paste before cursor

SHELL PROGRAM

Write First Shell Program

⇨ Shell script file is usually created with `.sh` extension.

❖ Step1: Create a file

Command:	Output:

`touch hello.sh`	`mohsin@X1-Yoga:~/os$ touch hello.sh`
	`mohsin@X1-Yoga:~/os$ ls`
	`hello.sh`
	`mohsin@X1-Yoga:~/os$`

❖ Step2: Open file using editor

Command:	Output:
`vi hello.sh`	

❖ Step3: First line of shell script

Command:	Output:
`#!/bin/bash`	
Note: Tells the system where **bash** is located.	

❖ Step4: Print text

Command:	Output:

`echo Hello World`
`or`
`echo "Hello World"`
	

Running First Program

❖ Step1: try running

Command:	Output:

`./hello.sh`	`mohsin@X1-Yoga:~/os$ ./hello.sh`
	`-bash: ./hello.sh: Permission denied`
	`mohsin@X1-Yoga:~/os$`

❖ Step2: If permission denied, check permissions

Command:	Output:

`ls -l hello.sh`	`mohsin@X1-Yoga:~/os$ ls -l hello.sh`
	`-rw-r--r-- 1 mohsin mohsin 29 Jan 13 23:16 hello.sh`
	`mohsin@X1-Yoga:~/os$`

❖ Step3: Give execute permission

Command:	Output:

`chmod u+x hello.sh`	`mohsin@X1-Yoga:~/os$ ls -l hello.sh`
	`-rw-r--r-- 1 mohsin mohsin 29 Jan 13 23:16 hello.sh`
	`mohsin@X1-Yoga:~/os$ chmod u+x hello.sh`
	`mohsin@X1-Yoga:~/os$ ls -l hello.sh`
	`-rwxr--r-- 1 mohsin mohsin 29 Jan 13 23:16 hello.sh`
	`mohsin@X1-Yoga:~/os$`

❖ Step4: Run script

Command:	Output:

`./hello.sh`
`or`
`bash hello.sh`
	`mohsin@X1-Yoga:~/os$ bash hello.sh`
	`Hello World`
	`mohsin@X1-Yoga:~/os$`

Shell Variables

- ⇨ No need to declare variables.
- ⇨ Use $ sign to access value.

❖ Types of Variables

1. **System Variables:**
   - Created by OS
   - Written in **CAPITAL letters**
   - Examples:
     - `$BASH` → bash location
     - `$BASH_VERSION` → bash version
     - `$HOME` → home directory
     - `$PWD` → current directory

2. **User Defined Variables:**

Correct Syntax:	Wrong Syntax:
`Name=Alex`	`Name = Alex  //No spaces allowed`

Using variable

Syntax:	Output:

`#!/bin/bash`
`Name=Mosin`
`echo $Name`
`echo My name is $Name`
	`mohsin@X1-Yoga:~/os$ bash hello.sh`
	`Mosin`
	`My name is Mosin`
	`mohsin@X1-Yoga:~/os$`

Shell Variable Rules:

- No data types (can store anything).
- No declaration needed.
- Variable name:
  - Cannot start with numbers
  - No spaces allowed

Comments in shell Program

➔ **Comments** are notes in the code that are ignored by shell during execution. They help explain the code for humans.

Types of Comments:

Single-line Comment: starts with #	**Multi-line Comment:** starts with :

`# This is a single-line comment`
`echo "Hello"  # This prints Hello`
	`: '`
	`This is a`
	`multi-line comment`
	`in a shell script`
	`'`
	`echo "Hi"`

Escape Characters

➔ Used to print special characters.

Example code:	**Output:**

`echo Hello \"World\"`	`mohsin@X1-Yoga:~/os$ bash hello.sh`
	`Hello "World"`
	`mohsin@X1-Yoga:~/os$`

Reading User Input

⇨ Shell scripts can **read input from the user** using the `read` command.

⇨ The value entered by the user is stored in a **variable**.

Syntax:
`read variable_name`

Example Code:

Example code:	**Output:**

`echo "Enter your name:"`
`read name`
`echo "Hello $name"`
	`Enter your name:`
	`Mohsin Ibna Hossain`
	`Hello, Mohsin Ibna Hossain`

⇨ Reading Multiple Inputs:

Syntax:

```
read variable1 variable2
```

Note: Input separated by **space**.

Example Code:

Example code:

```
echo "Enter a & b"
read a b
echo a:$a b:$b
```

Output:

```
Enter a & b:
10 20
a:10 b:20
```

Same Line Input

Example code:

```
read -p "Enter your age: " age
echo "Your age is $age"
```

Output:

```
Enter your age: 22
Your age is 22
```

Silent Input (Password)

Syntax:

```
read -s variable_name
```

Example code:

```
echo "Enter your Password"
read -s password
echo "Password received"
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
Enter your Password
Password received
mohsin@X1-Yoga:~/os$
```

Silent Input in same line:

Example code:	Output:

`read -sp "Enter password: " password`
`echo "Password received"`
	`mohsin@X1-Yoga:~/os$ bash hello.sh`
	`Enter password: Password received`
	`mohsin@X1-Yoga:~/os$`

Arithmetic Operators in Shell Programming

- **Arithmetic operators** are used to perform mathematical calculations in shell scripts.
- Bash cannot calculate directly.
- Shell supports arithmetic using:
  - `expr` or `$(expr)`
  - `$(( ))`

Syntax: `for` `expr`	Syntax: `for` `$(expr)`
`` `expr $a + $b` ``	`$(expr $a + $b)`
Note: Spaces are **mandatory** in `expr`.	

Common Arithmetic Operators:

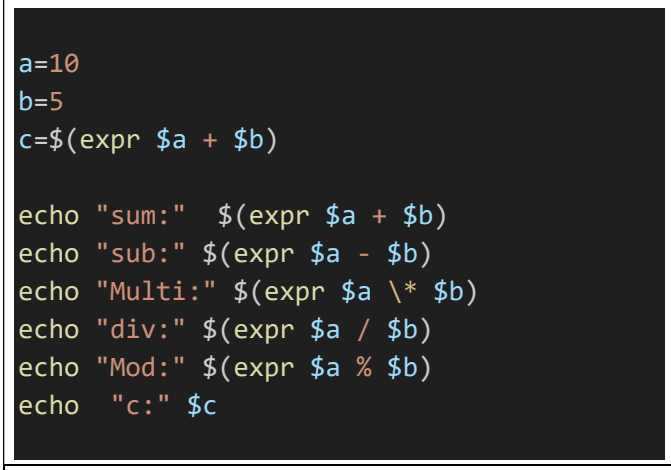
Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (remainder)

Example Code: Using `expr`

Example code:	Output:

`a=10`
`b=5`
`c=` `` `expr $a + $b` ``
 	
`echo "sum:" ` `` `expr $a + $b` ``
`echo "sub:" ` `` `expr $a - $b` ``
`echo "Multi:" ` `` `expr $a \* $b` ``
`echo "div:" ` `` `expr $a / $b` ``
`echo "Mod:" ` `` `expr $a % $b` ``
`echo "c:" $c`
	`mohsin@X1-Yoga:~/os$ bash hello.sh`
	`sum: 15`
	`sub: 5`
	`Multi: 50`
	`div: 2`
	`Mod: 0`
	`c: 15`
	`mohsin@X1-Yoga:~/os$`

❖ **Example Code:** Using `$(expr)`

Example code:	Output:
	

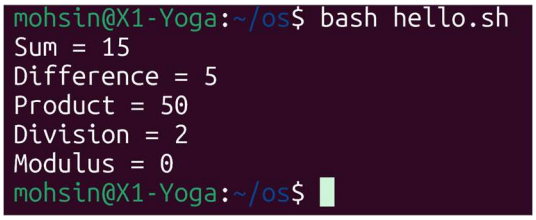
```
a=10
b=5
c=$(expr $a + $b)

echo "sum:"  $(expr $a + $b)
echo "sub:" $(expr $a - $b)
echo "Multi:" $(expr $a \* $b)
echo "div:" $(expr $a / $b)
echo "Mod:" $(expr $a % $b)
echo  "c:" $c
```

```
mohsin@X1-Yoga:~/os$ bash hello.sh
sum: 15
sub: 5
Multi: 50
div: 2
Mod: 0
c: 15
mohsin@X1-Yoga:~/os$
```

Note: in expr * must be escaped as *

❖ **Example Code:** Using `$(())`

Example code:	Output:
	

```
a=10
b=5

sum=$((a + b))
diff=$((a - b))
prod=$((a * b))
div=$((a / b))
mod=$((a % b))

echo "Sum = $sum"
echo "Difference = $diff"
echo "Product = $prod"
echo "Division = $div"
echo "Modulus = $mod"
```

```
mohsin@X1-Yoga:~/os$ bash hello.sh
Sum = 15
Difference = 5
Product = 50
Division = 2
Modulus = 0
mohsin@X1-Yoga:~/os$
```

Relational Operators in Shell

- **Relational operators** are used to **compare numeric values** in shell scripts.
- Typically used with **`if` statements** or **loops**.

Common Relational Operators:

Operator	Or use	Meaning
`-eq`	`==`	Equal to
`-ne`	`!=`	Not equal to
`-gt`	`>`	Greater than
`-lt`	`<`	Less than
`-ge`	`>=`	Greater than or equal to
`-le`	`<=`	Less than or equal to

Syntax:

Correct Syntax:	Wrong Syntax:
`[ $a -eq $b ]`	`[$a==$b]`
Note: Use `[ ]` with relational operators for **numeric comparisons**	

Example1:

Example code:

```
a=10
b=5

if [ $a -eq $b ]; then
  echo "a is equal to b"
else
  echo "a is not equal to b"
fi
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
a is not equal to b
mohsin@X1-Yoga:~/os$
```

Example2:

Example code:

```
a=10
b=5

if [ $a > $b ]; then
  echo "a is greater than b"
fi
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
a is greater than b
mohsin@X1-Yoga:~/os$
```

String Operators

- **String operators** are used to **compare or check properties of strings** in shell scripts.
- Typically used in **`if` statements**.

Common String Operators:

Operator	Meaning
`=`	Check if two strings are equal
`!=`	Check if two strings are not equal
`-z`	True if the string length is zero
`-n`	True if the string length is not zero

Syntax:

Correct Syntax:	Wrong Syntax:
`[ $str1 = $str2 ]`	`[$str1==$dtr2]`

Note: Use `[ ]` with string operators for **string comparison**.

Example1: Check Equality

Example code:

```
str1="Hello"
str2="World"

if [ $str1 = $str2 ]; then
  echo "Strings are equal"
else
  echo "Strings are not equal"
fi
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
Strings are not equal
mohsin@X1-Yoga:~/os$
```

Example2: Check if String is Empty

Example code:

```
str=""
if [ -z "$str" ]; then
  echo "String is empty"
fi
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
String is empty
mohsin@X1-Yoga:~/os$
```

Floating Point Calculation

- **Shell by default supports only integer arithmetic**.
- To perform **floating-point calculations**, we use `bc` (Basic Calculator) or `awk`.

❖ Example1: Using `bc`

Example code:	Output:

`num1=20.5`
`num2=10`
`echo $num1+$num2 | bc`
	`mohsin@X1-Yoga:~/os$ bash hello.sh`
	`30.5`

❖ Example2: Store Result

Example code:	Output:

`num1=20.5`
`num2=10`
`num3=$(bc <<< $num1+$num2)`
 	
`echo $num3`
	`mohsin@X1-Yoga:~/os$ bash hello.sh`
	`30.5`

Control Flow Statements

- **Control flow statements** are used to control the **order in which commands are executed** in a shell script.
- Common control flow statements include **if-else, case, loops (for, while, until), break, continue**.

if Statement

- Used to execute statements based on a condition.

❖ Syntax:

Method1:

```
if [ condition ]
then
    statement
else
    statement
fi
```

Method2:

```
if [ condition ]; then
  commands
else
  commands
fi
```

Note: There **must be space** before and after the condition.

- `if [condition]` → WRONG
- `if [ condition ]` → CORRECT

❖ Example1:

Example code:

```
a=6
b=5
if [ $a>$b ]
then
    echo hi
else
    echo hello
fi
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
hi
mohsin@X1-Yoga:~/os$
```

if – else if Statement

⇨ Used when **multiple conditions** are checked.

❖ Syntax:

Method 1:

```
if [ condition ]
then
    statement
elif [ condition ]
then
    statement
else
    statement
fi
```

Method 2:

```
if [ condition ]; then
  commands
elif [ condition ]; then
  commands
else
  commands
fi
```

❖ Example1:

Example code:

```
count=10
if [ $count -eq 9 ]
then
    echo "Hello World"
elif [ $count -le 6 ]
then
    echo "Hello OS"
else
    echo "Hello Mohsin"
fi
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
Hello Mohsin
mohsin@X1-Yoga:~/os$
```

case Statement

- ⇨ Used instead of **multiple else-if conditions**
- ⇨ Cleaner and easier for multi-condition checking.

❖ Syntax:

Correct Syntax:

```
case value in
  pattern1) commands ;;
  pattern2) commands ;;
  *) default commands ;;
esac
```

Note: * works as **default case**

❖ Example1:

Example code:

```
read -p "Enter a number (1-3): " num
case $num in
  1) echo "One" ;;
  2) echo "Two" ;;
  3) echo "Three" ;;
  *) echo "Invalid number" ;;
esac
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
Enter a number (1-3): 3
Three
mohsin@X1-Yoga:~/os$
```

Logical Operators

⇨ Used to combine **multiple conditions**.

OR Operator

⇨ Condition is true if **any one** condition is true.

❖ **Syntax:**

Method 1:

```
if [ $val -gt 18 ] || [ $val -lt 30 ]
then
    statement
else
    statement
fi
```

Method 2:

```
if [ $val -gt 18 -o $val -lt 30 ]
then
    statement
else
    statement
fi
```

Method 3: (Recommended)

```
if [[ $val -gt 18 || $val -lt 30 ]]
then
    statement
else
    statement
fi
```

❖ **Example1:**

Example code:

```
a=10
b=-5

if [ $a -lt 0 ] || [ $b -lt 0 ]; then
  echo "At least one number is negative"
fi
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
At least one number is negative
mohsin@X1-Yoga:~/os$
```

AND Operator

⇨ Condition is true only if **all conditions** are true.

❖ Syntax:

Method 1:

```
if [ $val -gt 18 ] && [ $val -lt 30 ]
then
    statement
else
    statement
fi
```

Method 2:

```
if [ $val -gt 18 -a $val -lt 30 ]
then
    statement
else
    statement
fi
```

Method 3: (Recommended)

```
if [[ $val -gt 18 && $val -lt 30 ]]
then
    statement
else
    statement
fi
```

❖ Example1:

Example code:

```
a=10
b=5

if [ $a -gt 0 ] && [ $b -gt 0 ]; then
  echo "Both numbers are positive"
fi
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
Both numbers are positive
mohsin@X1-Yoga:~/os$
```

Loop in Shell Scripting

⇨ A loop is used to **execute a set of commands repeatedly** until a condition is satisfied.

❖ Types of Loops in Shell Scripting:

⇨ There are **four (4) types** of loops:

1. **While Loop**
2. **For Loop**
3. **Select Loop**
4. **Until Loop**

While Loop

⇨ The **while loop** executes statements **as long as the condition is true**.

❖ Syntax:

Method 1:

```
while [ condition ]
do
    statement
    statement
done
```

❖ Example1:

Example code:

```
n=1
while [ $n -le 10 ]
do
    echo "$n"
    n=$((n+1))    # or ((n++)) or ((++n))
done
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
1
2
3
4
5
6
7
8
9
10
mohsin@X1-Yoga:~/os$
```

For Loop

⇨ The **for loop** is used to iterate over a list of values, files, or command outputs.

❖ Syntax:

Method 1: List-based For Loop

```
for VARIABLE in 1 2 3 4 5
do
    statement
done
```

Method 2: Range-based For Loop (Bash 4+)

```
for number in {1..100}
do
    echo $number
done
```

Method 3: C-style for loop

```
for (( i=0; i<10; i++ ))
do
    echo $i
done
```

❖ Example1:

Example code:

```
for number in 1 2 3 4 5
do
    echo $number
done
```

Output:

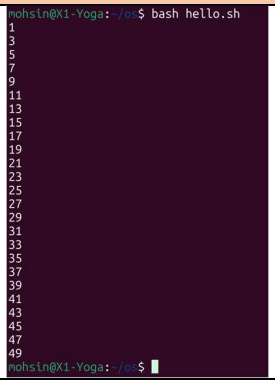
```
mohsin@X1-Yoga:~/os$ bash hello.sh
1
2
3
4
5
mohsin@X1-Yoga:~/os$
```

❖ Example2:

Example code: Increment by 2

```
for number in {1..50..2}
do
    echo $number
done
```

Output:



For Loop With Commands

⇨ Used to loop through commands.

❖ **Example1:**

Example code:

```
for command in ls pwd date
do
    echo "Command Name: $command"
    echo "Command Output:"
    $command
done
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
Command Name: ls
Command Output:
5  5]  hello.sh
Command Name: pwd
Command Output:
/home/mohsin/os
Command Name: date
Command Output:
Wed Jan 14 03:37:37 +06 2026
mohsin@X1-Yoga:~/os$
```

Note:

- With `echo`, the command **will not execute**
- Without `echo`, the command **will execute**

Select Loop With Case

⇨ The **select loop** is used to create **menu-driven programs**.

❖ **Syntax:**

Method 1:

```
select variable in alal dulal rahim
do
    case $variable in
        alal) echo "Alal is selected" ;;
        dulal) echo "Dulal is selected" ;;
        rahim) echo "Rahim is selected" ;;
        *) echo "Default" ;;
    esac
done
```

Note:

- Takes input **infinitely**
- Press **Ctrl + C** to exit

Until Loop

⇨ The **until loop** runs **until the condition becomes true**.

❖ Syntax:

Method 1:

```
until [ condition ]
do
    statement
done
```

❖ Example1:

Example code:

```
a=0
until [ ! $a -lt 10 ]
do
    echo $a
    ((a++))
done
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
0
1
2
3
4
5
6
7
8
9
mohsin@X1-Yoga:~/os$
```

❖ Example2:

Example code:

```
i=1
until [ $i -gt 5 ]
do
    echo $i
    i=$((i+1))
done
```

Output:

```
mohsin@X1-Yoga:~/os$ bash hello.sh
1
2
3
4
5
mohsin@X1-Yoga:~/os$
```

Break and Continue

Break Statement

⇨ Used to **exit the loop immediately**.

❖ Example1:

Example code:	Output:
	

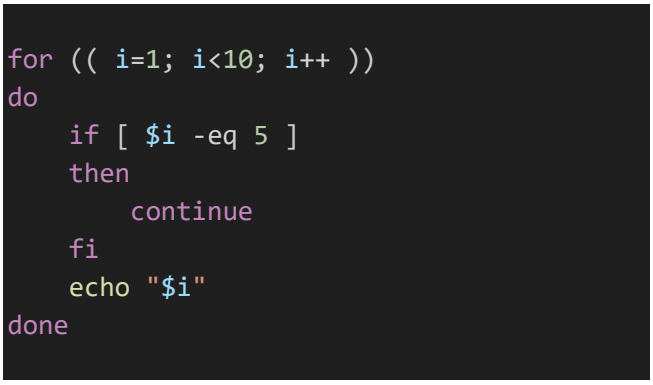
```
for (( i=1; i<10; i++ ))
do
    if [ $i -gt 5 ]
    then
        break
    fi
    echo "$i"
done
```

```
mohsin@X1-Yoga:~/os$ bash hello.sh
1
2
3
4
5
mohsin@X1-Yoga:~/os$
```

Continue Statement

⇨ Used to **skip the current iteration** and continue with the next one.

❖ Example1:

Example code:	Output:
	

```
for (( i=1; i<10; i++ ))
do
    if [ $i -eq 5 ]
    then
        continue
    fi
    echo "$i"
done
```

```
mohsin@X1-Yoga:~/os$ bash hello.sh
1
2
3
4
6
7
8
9
mohsin@X1-Yoga:~/os$
```

Practice Problem

Problem1: Write a shell script to **count the number of words** in the file **text.txt** using a **system variable**.

Answer:

```
#!/bin/bash
FILE=text.txt
wc -w $FILE
```

Problem2: Write a shell script to **display the hidden files** of a directory.

Answer:

```
#!/bin/bash
a="ls -a"
$a
```

Problem3: Write the output of the following script.

```
#!/bin/bash
for number in { 1..10..3}
do
    echo $number
done
```

Answer:

```
1
4
7
10
```

Problem4: Write a shell script to display the **calendar of the current month** using a system variable.

Answer:

```
#!/bin/bash
MONTH=cal
$MONTH
```

Problem5: Write a shell program to calculate the sum of the series [7+ 77 + 777 + 7777 ...] (up to n terms).

Answer:

```
#!bin/bash
read -p "Enter value n: " n
num=""
sum=0

for (( i=0; i<n; i++))
do
 num=${num}7
 sum=$(expr $sum + $num)
done
echo $sum
```

Problem6: Write a shell script to read **three integer numbers** from the user and find the **largest** among them.

Answer:

```
#!/bin/bash

read -p "Enter three number: " num1 num2 num3

if [ $num1 -gt $num2 ] && [ $num1 -gt $num3 ]
then
    echo $num1 is the largest number

elif [ $num2 -gt $num1 -a $num2 -gt $num3 ]
then
    echo $num2 is the largest number

else
    echo $num3 is the largest number
fi
```

Problem7: Write a shell script to display the following pattern:

```
#####
####
##
#
```

Answer:

```
#!/bin/bash

rows=5

for ((i=rows; i>=1; i--))
do
    if [ $i -eq 3 ]
    then
        continue
    fi

    for ((j=1; j<=i; j++))
    do
        echo -n "#"
    done
    echo
done
```

Problem8: Write a shell script to display the following pattern:

```
A
AA
AAA
AAAA
AAAAA
```

Answer:

```
#!/bin/bash

rows=5
for ((i=1; i<=rows; i++))
do
    for ((j=1; j<=i; j++))
    do
        echo -n "A"
    done
    echo
done
```